

Universidad de La Habana
Facultad de Matemática y Computación



Clasificación de similitud entre códigos de C#

Autor:

María de Lourdes Choy Fernández

Tutor:

Rodrigo García Gomez

Alberto Fernández Oliva

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

febrero de 2024

github.com/Chonyyy/Code_Similarity

A mi familia
A mis amigos
A mis profesores.

Agradecimientos

Quiero expresar mi más sincero agradecimiento a todas las personas que me han apoyado y acompañado durante el proceso de realización de esta tesis.

Agradezco a mi tutor de tesis, Rodrigo García, por su orientación, paciencia y valiosos comentarios que me han permitido enriquecer este trabajo. Su dedicación y conocimiento han sido fundamentales en el desarrollo de esta investigación.

Agradezco también a mis compañeros y amigos, quienes, con su apoyo y motivación, me ayudaron a superar los momentos más difíciles. Su comprensión y amistad han sido una constante fuente de inspiración.

A mi familia, por su amor incondicional y su respaldo constante. A mis padres, Lourdes Fernández y Arturo Choy, por siempre brindarme su confianza y apoyo inquebrantable en cada etapa de mi vida académica.

Finalmente, agradezco a todas las personas que, de alguna manera, contribuyeron al éxito de este proyecto, ya sea a través de sus conocimientos, su tiempo o su apoyo emocional.

Opinión del tutor

El fraude es un problema recurrente en la educación. Los profesores luchan constantemente contra las faltas morales tanto en exámenes como en proyectos y evaluaciones teóricas. Nuestra facultad no está exenta de este problema. La asignatura de Programación, de primer año de Ciencias de la Computación en MATCOM, tiene un sistema en el que el 50% de la calificación del estudiante depende de proyectos evaluativos. Es común que año tras años los profesores encontremos, a ojo, proyectos clonados que representan faltas éticas. Sobre este marco, la estudiante desarrolló un software que nos resultará altamente útil.

Un *software* capaz de detectar plagios en proyectos de C# tendrá dos claras aportaciones a nuestra carrera. En primer lugar, evitará que prácticas inmorales pasen desapercibidas. En segundo lugar, automatizará una tarea que consume valioso tiempo a nuestro claustro: Buscar estas prácticas amorales.

María hizo una amplia investigación del estado del arte actual y notó que las herramientas libres de costo para comparación de código en lenguaje C# (lenguaje que se usa en nuestra asignatura) eran ineficientes, casi inexistentes. Ella fue capaz de editar una gramática de C# 6.0 para convertirla en una 12.0. Luego utilizó esa gramática para extraer características de los AST de un amplio conjunto de códigos de entrenamiento. Finalmente utilizó estas características para entrenar un modelo clasificador de aprendizaje automático. Este trabajo demostró conocimientos en un gran abanico de campos técnicos como: Compilación, Inteligencia artificial y programación orientada a objetos.

Como tutores, estamos orgullosos de que María haya convertido una queja aleatoria de un profesor de programación: "Qué complicado estar buscando trampas por más de 100 proyectos", en un excelente trabajo investigativo.

Por todo lo anterior creemos que estamos en presencia de una excelente científica de la computación, que al igual que el trabajo, merece la máxima calificación.

Rodrigo García Gómez y Alberto Fernández Oliva

Resumen

Esta investigación aborda el problema de la detección de similitudes en código fuente de proyectos de C#. El proceso comienza con la extracción del Árboles de Sintaxis Abstracta (AST), lo que permite una representación estructural del código. A partir de los AST, se extraen características que describen elementos del código. Las características se agrupan en vectores que se utilizan como entrada de una Red Neuronal Siamesa. La red siamesa se entrena como un modelo clasificador de similitud capaz de detectar parejas de códigos de C# clonados.

Abstract

This research addresses the problem of detecting similarities in the source code of C# projects. The process begins with the extraction of Abstract Syntax Trees (AST), which provides a structural representation of the code. From the ASTs, features that describe elements of the code are extracted. These features are grouped into vectors, which are used as input for a Siamese Neural Network. The Siamese network is trained as a similarity classifier model capable of detecting pairs of cloned C# code.

Índice general

Introducción	10
1. Estudio del estado actual de la técnica	13
1.1. Introducción al Análisis de Similitud de Código	13
1.2. Técnicas Basadas en Árboles de Sintaxis Abstracta (AST) . .	14
1.3. Avances en Análisis de Similitud de Código	16
1.3.1. Integración del Aprendizaje Automático	17
1.3.2. Redes Neuronales Siamesas	18
1.4. Representaciones Vectoriales y Aprendizaje Profundo	19
1.5. Uso de ANTLR y Python para extracción de ASTs	20
1.6. Resumen	20
2. Extracción de <i>features</i> del AST	23
2.1. Modificaciones a la Gramática de ANTLR para Soporte de C# Actual	23
2.2. Extracción del AST	24
2.3. Extracción de features	26
3. Preparación del conjunto de entrenamiento (<i>dataset</i>)	31
3.1. Word2Vec	31
3.2. Vectores de características	32
3.3. Construcción del dataset	32
4. Redes Neuronales Siamesas para la Similitud de Código	35
4.1. Red Neuronal Siamesa	35
4.2. Implementación del Modelo	36
4.2.1. Subred Base	37
4.2.2. Construcción del Modelo Siamés	38

4.2.3. Función de Pérdida	39
4.2.4. Optimización	40
4.3. Proceso de Entrenamiento y Predicción	42
4.3.1. Entrenamiento	42
4.3.2. Predicción	42
5. Experimentos y resultados	44
5.1. Evaluación del Modelo	44
5.2. Experimentos	45
5.3. Validación de resultados	47
6. Conclusiones	50
Recomendaciones	51
Anexos	52
Bibliografía	79

Introducción

El análisis de similitud de código en Ciencias de la Computación se centra en la comparación de fragmentos de este para identificar similitudes y diferencias entre ellos. Este enfoque se utiliza en la detección de plagios y en la revisión de código, en sentido general.

Esta investigación surge de la necesidad de dotar a las instituciones educativas de herramientas que puedan analizar código con precisión. Al combinar técnicas de aprendizaje automático [23] y extracción de Árboles de Sintaxis Abstracta [11], se propone un enfoque para la detección de similitud de código. Este enfoque no sólo busca resolver los problemas de plagio en entornos académicos, sino también contribuir al entendimiento teórico del problema y ofrecer una base para futuras investigaciones en el área.

Motivación y justificación

En el contexto académico, la detección de similitud de código y el análisis de plagios representan un desafío en las evaluaciones docentes. Los profesores enfrentan la tarea de garantizar la originalidad en las soluciones de los estudiantes a los problemas de programación que se le presenta. Este problema no solo afecta la integridad académica, sino que también dificulta evaluar de manera justa el aprendizaje y la comprensión de conceptos.

El plagio de código puede presentarse mediante prácticas como la copia literal, el cambio de nombres de variables, la reorganización de bloques de código o la reescritura con estructuras equivalentes que mantengan la lógica de los algoritmos. Como resultado, la detección efectiva requiere sistemas que puedan identificar similitudes semánticas y estructurales, independientemente de las transformaciones que se realizan.

Formulación del problema

Formalmente el problema que este trabajo propone resolver es: dado un conjunto de proyectos computacionales en el lenguaje C#, encontrar parejas de éstos que sean similares entre sí. El ámbito del problema en esta investigación se encuentra dentro del dominio general de los problemas de Inteligencia Artificial, particularmente en los procesos de análisis de similitud y detección de patrones en conjuntos de datos estructurados. Este problema es relevante en múltiples áreas, como la detección de plagios en código fuente, la refactorización de *software*, la optimización de bases de código y la identificación de redundancias en sistemas de desarrollo colaborativo.

Hipótesis

Dado que se tiene un conjunto de proyectos de la asignatura de programación en lenguaje C#, entonces resultaría útil, para los profesores que evalúan esta asignatura, tener un software capaz de por cada pareja de proyectos dos a dos, extraer sus respectivos AST, compararlos, y clasificarlos de acuerdo a su similitud para descubrir posibles fraudes.

Objetivo General

El objetivo general de este trabajo de investigación es desarrollar un marco teórico para el análisis de similitud de código que combine la extracción de características mediante árboles de sintaxis abstracta (AST) con técnicas de aprendizaje automático para mejorar la precisión y la eficiencia en la detección de similitudes de código C#.

Objetivos Específicos

- I. **Extracción y Procesamiento de AST para la Comparación de Código**
Diseñar e implementar un sistema que utilice Árboles de Sintaxis Abstracta (AST) para analizar y comparar código o proyectos de C#.

II. Integrar Técnicas de Aprendizaje Automático para la Detección de Similitudes

Integrar técnicas de aprendizaje automático para mejorar la detección de similitudes en el código. Entrenar un modelo de aprendizaje supervisado utilizando un conjunto de datos conformado por proyectos de C#.

III. Desarrollar y Validar una Herramienta Práctica

Desarrollar una herramienta de *software* que implemente las técnicas desarrolladas, y sea fácil de usar por desarrolladores y educadores.

Propuesta de solución

Se propone extraer los ASTs de códigos de C#, convertirlos en vectores de características y utilizarlos como entrada a un modelo de Red Neuronal Siamesa. El modelo asignará valores entre 0 y 1 a parejas de códigos según su similitud (siendo más parecido mientras más se acerca a 1) y se utilizará en un *software* clasificador. La red siamesa se entrenará utilizando un conjunto de códigos generados por Inteligencias Artificiales generativas y otro conjunto compuesto por copias manuales de proyectos de C#. De este modo se obtendrá una herramienta práctica, que se pueda integrar en entornos de desarrollo reales.

La validación de la herramienta se llevará a cabo en escenarios prácticos, como la detección de plagios en tareas de programación de código. Esto no solo demostrará la efectividad de las técnicas implementadas, sino también garantizará que la herramienta sea útil para los usuarios finales.

El presente documento está organizado en 6 capítulos. En el capítulo 1 se describen los elementos fundamentales del problema de similitud de código. En el capítulo 2 se describe la extracción de características del AST. Por su parte en el capítulo 3 se muestra la preparación del conjunto de entrenamiento y en el capítulo 4 se describe el modelo de red neuronal utilizado para este problema. En el capítulo 5 se exponen los experimentos y resultados alcanzados y finalmente en el capítulo 6 se exponen las conclusiones del trabajo y se presentan, a modo de recomendaciones, las líneas futuras que dan continuidad a la presente investigación.

Capítulo 1

Estudio del estado actual de la técnica

1.1. Introducción al Análisis de Similitud de Código

El análisis de similitud de código tiene sus inicios en la década de 1970 [31]. Se utiliza para la detección de plagios, la revisión de código y como herramientas de asistencia a clases. En esta sección se revisan los desarrollos históricos, metodologías y tecnologías relacionadas con este tema, así como los avances recientes en este ámbito.

Los orígenes del análisis de similitud de código se remontan a los años 70 con los primeros algoritmos de coincidencia de cadenas de texto, cuando se buscaban métodos para detectar plagios en tareas de programación. Algoritmos como Knuth-Morris-Pratt (KMP) [31] y Boyer-Moore [14] se utilizaron inicialmente para comparar secuencias de texto, sentando las bases para tareas posteriores relacionadas con la comparación de código fuente la comparación de código fuente.

En los 90, herramientas como MOSS (Measure of Software Similarity) [4] fueron un primer paso dentro de este ámbito. MOSS normaliza el código, transformándolo en una representación estándar que facilita su comparación y reduce variaciones superficiales sin alterar su significado. En este proceso, MOSS elimina comentarios, renombra variables y unifica ciertos aspectos estructurales del código para detectar similitudes más allá de diferencias menores en la escritura. Esta normalización permite iden-

tificar similitudes estructurales y lógicas incluso frente a modificaciones superficiales, como cambios en los nombres de variables y funciones, alteraciones en la indentación o formato, reordenamiento de bloques de código sin afectar la lógica del programa, inserción o eliminación de comentarios, e incluso, reformulaciones sintácticas que no alteran la semántica del código.

1.2. Técnicas Basadas en Árboles de Sintaxis Abstracta (AST)

Originalmente, fueron utilizadas en los 80 para ser usados en el desarrollo de compiladores [3], con posterioridad a esto, los AST comenzaron a usarse para el análisis y la transformación de código. A través de ellos, se pueden realizar comparaciones basadas en la estructura lógica y sintáctica del código, lo cual supera las limitaciones de las comparaciones textuales. Estas comparaciones más avanzadas se enfocan en el análisis de la estructura interna del programa, a través del uso de los AST y el análisis de flujos de control y las relaciones entre las diferentes partes del código, en lugar de simplemente comparar las secuencias de texto. Este enfoque permite identificar similitudes entre fragmentos de código que, aunque puedan diferir en su sintaxis o formato, mantienen la misma lógica subyacente, como ocurre con las reescrituras de código o cambios en los identificadores.

Un AST es una representación jerárquica y estructurada de un código fuente [30]. Cada nodo del árbol representa una construcción en un lenguaje de programación, tales como operadores, declaraciones o expresiones. Esta representación abstracta facilita el análisis del código al ignorar detalles superficiales como el formato o los comentarios. Un ejemplo de un fragmento de AST se muestra en la figura 1.1.

En I. Baxter et al. propusieron métodos que utilizan hashing¹ para de-

¹Hashing es el proceso de transformar datos de longitud variable en un valor de longitud fija usando una función hash. Este valor, conocido como hash, es único para cada conjunto de datos, lo que permite comparar rápidamente información sin necesidad de analizar los datos completos. Se utiliza en estructuras de datos (como tablas hash), verificación de integridad (para detectar cambios en archivos) y seguridad (por ejemplo, en contraseñas). La principal característica del hashing es que pequeños cambios en los datos de entrada producen resultados muy diferentes.

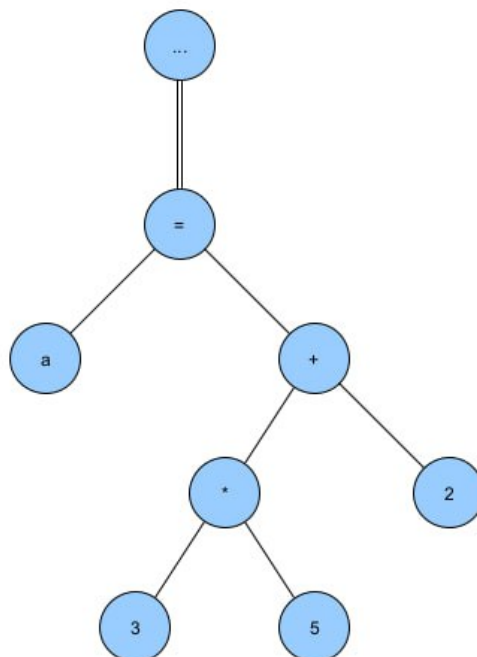


Figura 1.1: La imagen muestra un subgrafo que representa una ecuación $a = 3 * 5 + 2$

tectar clones exactos [11] y casi coincidentes mediante subárboles de AST. Aunque efectivos, enfrentan desafíos como falsos positivos en la detección de clones complejos.

B. N. Pellin utilizó una Máquina de Soporte Vectorial (SVM) y los AST para clasificar autoría en código fuente [50].

En investigaciones recientes, se enriquecen los métodos de árboles de sintaxis abstracta (AST) con funciones de kernel para mejorar su capacidad descriptiva en la detección de clones y similitudes. Las funciones de kernel son técnicas matemáticas utilizadas en aprendizaje automático, especialmente en métodos como las Máquinas de Soporte Vectorial (SVM). A partir de esto, se pueden capturar patrones complejos, mejorando la precisión en la detección de similitudes, incluso, en casos donde las estructuras pueden diferir en términos de sintaxis o formato. Por ejemplo, Wang et al. [58] introdujo un enfoque que utiliza un kernel de árbol ponderado para identificar plagios en código fuente. Este método aumenta la precisión

al considerar las relaciones jerárquicas entre los nodos.

Por otro lado, la comparación de subárboles mediante la subsecuencia común más larga (LCS) es útil para detectar similitudes estructurales en algoritmos [54]. Este enfoque permite identificar patrones recurrentes cuando el código presenta modificaciones superficiales.

El análisis basado en AST no se limita a detectar similitudes textuales, sino que también abarca similitudes semánticas y estructurales. Estas capacidades lo convierten en una herramienta eficaz para analizar la lógica subyacente del código, aún en casos donde se realizan modificaciones para dificultar la detección directa.

1.3. Avances en Análisis de Similitud de Código

Los avances recientes en el análisis de similitud de código utilizan técnicas como la distancia de Levenshtein [56], que miden las modificaciones necesarias para transformar un fragmento de código en otro. También se emplean modelos de representación vectorial como Word2Vec y Code2Vec [6], que convierten el código en vectores para comparar similitudes. Estas técnicas utilizan el AST, que representan la estructura lógica del código, lo que permite detectar similitudes en la lógica del programa más allá de la sintaxis.

Entre las técnicas basadas en distancias de edición, destaca el uso de la distancia de Levenshtein, que es una métrica que calcula el número mínimo de operaciones (inserciones, eliminaciones o sustituciones de caracteres) necesarias para transformar una cadena de texto en otra. Esto permite detectar clones que presentan cambios significativos en su estructura, como la reordenación de bloques de código o cambios en los identificadores, pero que mantienen la misma lógica [56]. Por otro lado, la distancia de Damerau-Levenshtein [46] es una extensión de la distancia de Levenshtein que extiende este enfoque al análisis estructural del código mediante el empleo de AST y utiliza no solo las operaciones básicas de edición, sino también la reordenación de caracteres como una operación válida.

En cuanto a los modelos de representación vectorial², propuestas recientes como Code2Vec [6] y Code2Seq [5] se utilizan en el análisis de simi-

²Los modelos de representación vectorial son técnicas que convierten datos, como texto o código, en vectores numéricos en un espacio multidimensional.

litudes de código al transformar el código fuente en vectores que encapsulan tanto información sintáctica como semántica. Estos modelos emplean técnicas adaptadas del procesamiento de lenguaje natural (NLP), como las utilizadas en la representación de palabras y frases mediante modelos de embeddings. En el contexto de código, adaptan estas técnicas para representar fragmentos de código como vectores, utilizando las estructuras de árboles de sintaxis abstracta (AST). En lugar de palabras o frases, se trabaja con caminos entre nodos en el AST, lo que permite capturar relaciones jerárquicas y secuenciales dentro del código fuente.

1.3.1. Integración del Aprendizaje Automático

Desde 2010, el aprendizaje automático contribuye a mejorar la precisión del análisis de similitud de código. El uso de redes neuronales recurrentes (RNN) [28] y convolucionales (CNN) [34] generan representaciones vectoriales que capturan patrones.

Por otro lado, la detección de similitudes mediante el uso de clustering y métricas como la similitud coseno se exploran en *Code Similarity Detection Technique Based on AST Unsupervised Clustering Method* [9]. Este enfoque combina métodos de agrupamiento o clustering con la estructura AST. El uso de técnicas no supervisadas en este contexto permite identificar patrones similares sin necesidad de etiquetar los datos previamente.

En [29] se presenta un estudio donde se destaca el uso de modelos de aprendizaje automático para detectar el código que generan herramientas como ChatGPT [16] en entornos académicos.

Los modelos más recientes [57] también exploran la integración de redes neuronales de grafos (Graph Neural Networks, GNN), que son técnicas de aprendizaje profundo que operan sobre estructuras de grafos, como las representaciones gráficas del código. Estos modelos permiten capturar relaciones no lineales y dependencias complejas entre los componentes del código. La integración con árboles de sintaxis abstracta (AST) permite combinar la representación estructural jerárquica del código con la capacidad de las GNN de capturar interacciones a nivel de grafo, lo que mejora la detección de patrones y similitudes en el código fuente.

Como se detalla en [57], este estudio implementa redes neuronales de grafos (GNN) y las combina con AST para mejorar la detección de clones de código. Con un 96% de precisión, este modelo aprovecha la represen-

tación gráfica del flujo de información en el código fuente para identificar similitudes estructurales profundas que otras técnicas podrían pasar por alto.

Otras investigaciones se centran en la detección de vulnerabilidades en el código mediante similitud estructural. Un ejemplo de esto se presenta en [8], que emplea la similitud coseno³ entre ASTs para identificar cercanías entre funciones en el código, y el uso de un enfoque de redes neuronales profundas (Deep Neural Networks, DNN)⁴ para mapear estas funciones a patrones conocidos de replicación.

1.3.2. Redes Neuronales Siamesas

Las redes neuronales siamesas son un tipo de arquitectura de red diseñada para aprender similitudes entre pares de entradas que son muestras relacionadas o similares. Por ejemplo imágenes, fragmentos de texto, o secuencias de código. Originalmente propuestas por Bromley y LeCun [15], estas redes constan de dos ramas idénticas que comparten los mismos parámetros. Cada rama procesa una de las entradas y genera una representación vectorial de alto nivel, la cual captura las características abstractas o esenciales del dato de entrada.

Una representación vectorial de alto nivel es una proyección del dato de entrada en un espacio de características latentes, que son representaciones abstractas de los datos que capturan la información más importante y significativa de los mismos, pero de forma comprimida y en un espacio de mayor nivel, donde se enfatizan las relaciones relevantes para la tarea específica como por ejemplo similitud, clasificación o agrupamiento y se descartan detalles irrelevantes [43]. En el contexto de las redes neuronales siamesas, estas representaciones permiten comparar entradas al evaluar su similitud o disimilitud mediante métricas específicas. Estas métricas, como la distancia euclidiana [19], la similitud del coseno [36] o la distancia L1 [13], identifican relaciones semánticas y estructurales entre los datos, incluso cuando no son idénticos textualmente [15,18].

³La similitud del coseno es una métrica utilizada para medir la similitud entre dos vectores en un espacio vectorial.

⁴Son una clase de modelos de aprendizaje automático que están compuestos por múltiples capas de neuronas, lo que les permite aprender representaciones complejas de los datos.

En [32] se introduce la arquitectura siamesa como un enfoque para la solución de problemas de comparación entre pares de entradas (imágenes, fragmentos de texto o secuencias de código). Este tipo de redes es especialmente útil en escenarios de aprendizaje de pocas muestras (se refiere a la capacidad de entrenar un modelo con un número muy reducido de ejemplos) en lugar de grandes cantidades de datos etiquetados. Las redes siamesas aprenden una métrica de similitud procesando pares de datos a través de dos ramas idénticas con pesos compartidos, lo que permite que la red compare las entradas y determine si son similares o no. Este enfoque reduce la necesidad de grandes conjuntos de datos etiquetados, los cuales son costosos y difíciles de obtener en muchas aplicaciones [32]. Su capacidad para generalizar con pocos ejemplos las hace ideales para tareas como la detección de similitud de código, donde puede ser difícil y costoso etiquetar grandes cantidades de código de manera manual.

En investigaciones recientes, como [10], esta arquitectura se utiliza junto con redes convolucionales (CNN) [33] para analizar similitudes semánticas, logrando precisiones de hasta un 90 %.

Por otro lado, en [35], se combinan redes siamesas con modelos de embeddings de palabras como Word2Vec para mejorar la representación de tokens en el análisis de similitud.

1.4. Representaciones Vectoriales y Aprendizaje Profundo

Word2Vec se utiliza para generar representaciones vectoriales de fragmentos de código, a partir de lo cual se pueden capturar relaciones semánticas complejas [43]. Las representaciones vectoriales permiten realizar comparaciones entre los fragmentos de código, pues se transforman en puntos dentro de un espacio vectorial. Estas representaciones capturan tanto la estructura lógica como los patrones semánticos.

Desde 2013, se han empleado redes neuronales profundas (DNN) para aprender incrustaciones vectoriales [43] de código. [27].

1.5. Uso de ANTLR y Python para extracción de ASTs

ANTLR (ANother Tool for Language Recognition) [51] es una herramienta que crea analizadores sintácticos y léxicos para lenguajes de programación, mediante la especificación de una gramática formal, que describe la estructura sintáctica de dicho lenguaje. A partir de esta gramática, ANTLR genera el código necesario para reconocer, analizar y manipular el código fuente [47].

Se utiliza ANTLR debido a su versatilidad y eficiencia para trabajar con diferentes lenguajes de programación, especialmente en el caso de C#. Una de las ventajas de ANTLR es su compatibilidad con múltiples lenguajes de programación, lo que permite generar analizadores para Python, Java, JavaScript, C#, entre otros. Al generar los analizadores en Python, se aprovecha la simplicidad y flexibilidad de este lenguaje, ampliamente utilizado en el campo de Machine Learning. Python, además, es un lenguaje que se caracteriza por su sintaxis clara y su extensa librería de herramientas para análisis y procesamiento de datos: NumPy [26], pandas [41], scikit-learn [49], TensorFlow [2], y PyTorch [48], por tal motivo, se utiliza para implementar modelos de Machine Learning [41].

1.6. Resumen

La tabla 1.1 muestra estudios sobre la detección de similitudes de código utilizando técnicas de **Machine Learning**, extracción de **AST**, y otros modelos en diferentes lenguajes de programación. Sin embargo, al observar los lenguajes de programación que se utilizan, se destaca que **C#** es un lenguaje con poca representación si se compara con otros como **Java**, **Python** o **C++**.

De los estudios que se mencionan, solo 2 sirven para comparar código de **C#**:

- **MOSS** (una herramienta para la detección de similitudes de código) soporta **C#**. Este *software* usa técnicas de normalización de código: eliminar comentarios y renombrar variables. No usa técnicas de **Machine Learning**. MOSS fue creado en 1994 y en la actualidad todavía es una herramienta popular, pero no se actualiza con regularidad [4].

Tabla 1.1 Estudio sobre la detención de similitudes de código utilizando técnicas de Machine Learning, extracción de AST y otros modelos, en diferentes lenguajes de programación

Paper	Machine Learning	AST	Modelo	Accuracy	Precisión	Recall	Lenguaje de Programación	Versión de C
Enhanced Semantic Similarity Detection of Program Code Using Siamese Neural Network	TRUE	TRUE	Redes Neuronales Siamesas, CNN	(67%-88%)	90%	95%	C++	
Code Similarity Detection Technique Based on AST Unsupervised Clustering Method	TRUE	TRUE	Unsupervised Clustering	0.8549-0.9043			Python	
Detecting ChatGPT-Generated Code Submissions in a CS1 Course Using Machine Learning Models	TRUE	TRUE	SVM, XGBoost, code2vec, ASTNN, and SANN	97%	99%	97%	Python	
Detecting Code Clones with Graph Neural Network and Flow-Augmented Abstract Syntax Tree	TRUE	TRUE	Graph Neural Networks (GNN)	96%	94%	95%	Java	
Asteria: Deep Learning-based AST-Encoding for Cross-platform Binary Code Similarity Detection	TRUE	TRUE	Twin Network with Tree-LSTM and softmax				Binario	
JAVA SOURCE CODE SIMILARITY DETECTION USING SIAMESE NETWORKS	TRUE	TRUE	Recursive Neural Networks, Siamese Neural Networks		100%	99%	Java	
Modeling Functional Similarity in Source Code with Graph-Based Siamese Network	TRUE	TRUE	Graph Neural Networks				Java	
Clone Detection Using Abstract Syntax Suffix Trees	FALSE	TRUE	None				(Java, C)	
MOSS: Measure of Software Similarity	FALSE	FALSE	Normalización de código (eliminación de comentarios, renombrado de variables)				(C, C++, Java, C, Python, Visual Basic, Javascript, FORTRAN, ML, Haskell, Lisp, Scheme, Pascal, Modula2, Ada, Perl, TCL, Matlab, VHDL, Verilog, Spice, MIPS assembly, a8086 assembly, a8086 assembly, HCL2.)	
JPlag: A System for Detecting Software Plagiarism	FALSE	TRUE	Tokenización, Clustering				C, Java, C++, Python	6.0

- **JPlag** [22] utiliza una versión desactualizada de **C# 6.0**. Se creó en 1996 y su última actualización fue en **2015**. No emplea técnicas de análisis de aprendizaje supervisado como redes neuronales. En cambio usa técnicas de clustering [22].

El análisis anterior muestra que existen pocas herramientas que detecten similitudes de código en **C#**, y ninguna en una versión actualizada del

lenguaje. Esto sugiere que este lenguaje no ha sido explorado con la misma profundidad que otros en el contexto de la comparación de código. Esta tesis propone un *software* capaz de analizar **C# 12** y que además combina técnicas de Aprendizaje Automático Supervisado con comparaciones de ASTs. Esto representa su principal contribución.

Capítulo 2

Extracción de *features* del AST

En el análisis de similitud de código y detección de patrones, es necesario extraer características relevantes que capturen la estructura y el comportamiento del código para permitir a los modelos de análisis identificar similitudes y patrones. Estas características, se conocen como *features*, y representan los aspectos más importantes de los datos, lo que permite analizar y comparar fragmentos de código de manera efectiva. Una de las técnicas más comunes para extraer estas características es el uso de Árboles de Sintaxis Abstracta (AST), que representan la estructura sintáctica de un fragmento de código en forma de árbol. En este capítulo se detalla la modificación de la gramática de ANTLR así como todo lo referido a la extracción del Árbol de Sintaxis Abstracta (AST) de los códigos y a la extracción de los *features* de dichos árboles.

2.1. Modificaciones a la Gramática de ANTLR para Soporte de C# Actual

ANTLR (Another Tool for Language Recognition) cuenta con una gramática basada en C# 6.0. Esta gramática presentaba limitaciones significativas al trabajar con el código moderno (o sea, código que utilice versiones recientes del lenguaje). Se generan errores durante el análisis sintáctico y se genera un árbol de sintaxis abstracta (AST) incompleto. Por tal motivo, se decide rabajar con una versión más actualizada del lenguaje C# 12.0 (introducida en noviembre de 2023) para reflejar las características que pre-

sentan versiones posteriores a C# 6. En los anexos se describen los cambios realizados a la gramática en dichas versiones, según lo reflejado en la documentación correspondiente de Microsoft [42] y las modificaciones que, como parte de este trabajo, se han realizado para actualizar la misma.

2.2. Extraccion del AST

ANTLR convierte una gramática de lenguaje en un código a partir del cual se puede generar un árbol de sintaxis. A continuación, se describen los pasos que se siguen para generar un AST a partir de un código fuente, que utiliza un proceso de análisis léxico y sintáctico.

- I. **Definición de la Gramática:** primero, se define la gramática del lenguaje en un archivo `.g4` que expresa la estructura de un lenguaje mediante una serie de reglas léxicas y sintácticas. Es el núcleo de la herramienta ANTLR y sirve como una especificación formal del lenguaje de programación que se desea analizar o procesar. Estas reglas se emplean para descomponer el texto de entrada en componentes que ANTLR puede reconocer y procesar.
 - Las **reglas léxicas (*tokens*)** se encuentran en el archivo *CSharpLexer.g4*. Estas se encargan de identificar los componentes más simples del lenguaje: palabras clave, identificadores, operadores, números, comillas, entre otros. Los *tokens* son las unidades mínimas de información con las que trabaja el analizador léxico, y cada uno de ellos se representa como una secuencia de caracteres que ANTLR puede reconocer.
 - Las **reglas sintácticas** se encuentran en el archivo *CSharpParser.g4*. Describen cómo los *tokens* del *lexer* se combinan en estructuras válidas dentro del lenguaje. Definen cómo los *tokens* se agrupan y organizan en expresiones, sentencias, funciones o cualquier otra estructura del lenguaje. Por ejemplo, una regla sintáctica puede definir cómo se estructura una asignación de variable, estableciendo que una asignación se compone de un identificador, seguido de un operador de asignación y una expresión que puede ser un valor o una operación.

- II. **Generación del Lexer y el Parser:** después de definir la gramática en un archivo `.g4`, ANTLR genera automáticamente las clases necesarias para implementar el *lexer* (analizador léxico) y el *parser* (analizador sintáctico) en Python. Como parte de este proceso, ANTLR también genera un árbol de sintaxis básico *Parse Tree*, el cual representa la estructura jerárquica del código fuente que se analiza, incluyendo todos los *tokens* y reglas gramaticales definidos en la gramática. Este árbol es fundamental para analizar el flujo y la estructura del programa, sirviendo como base para la creación de un Árbol de Sintaxis Abstracta (AST).

Se utilizó ANTLR versión 4.13.2, que procesa los archivos `.g4` y genera las clases necesarias en Python. De igual forma, ANTLR automatiza el proceso de análisis léxico y sintáctico, facilitando la creación de herramientas como compiladores y analizadores de código.

- III. **Análisis del Código o Texto de Entrada:** El lexer y parser se utilizan para procesar el código fuente o texto de entrada.

```
input_stream = FileStream(file_path, encoding='utf-8')
```

Este análisis comienza con el lexer, que divide el texto en tokens que siguen las reglas léxicas que se definen en la gramática.

```
lexer = CSharpLexer(input_stream)
tokens = CommonTokenStream(lexer)
```

Luego, el parser organiza estos tokens en una estructura jerárquica que refleja la gramática sintáctica del lenguaje. Este proceso de análisis genera un árbol de sintaxis, también conocido como un árbol de derivación, que representa la estructura jerárquica del código según las reglas definidas en el archivo `.g4`. Cada nodo del árbol corresponde a una regla de la gramática, lo que permite una representación estructurada del código.

```
parser = CSharpParser(tokens)
```

- IV. **Creación del Árbol de Sintaxis Abstracta (AST):** ANTLR no solo genera el árbol de sintaxis básico, sino que también crea un Árbol de Sintaxis Abstracta (AST), mediante la invocación de *compilation_unit()* que corresponde al nodo raíz del AST. A diferencia del

árbol de sintaxis, el AST elimina detalles como los símbolos delimitadores o ciertos tokens que no aportan al análisis lógico del programa. El AST conserva la estructura lógica y semántica del código, organiza las relaciones jerárquicas entre elementos como expresiones, declaraciones, y bloques de control. Esto permite realizar análisis, como la extracción de características o la transformación del código.

```
tree = parser.compilation_unit()
```

- V. **Recorridos y Transformaciones en el AST:** Después de construir el AST, se utilizó el listener `CSharpParserListener` de ANTLR para recorrer el árbol y extraer información relevante del código fuente, que se explica en la próxima sección. Este listener recorre el AST y maneja cada nodo de acuerdo con la lógica que se define en el código.

2.3. Extraccion de features

Se implementó una clase llamada **FeatureExtractorListener**, que extiende la funcionalidad de `CSharpParserListener` para analizar el código fuente en C#. A continuación, se describe del proceso de extracción de características y la importancia de cada una.

I. Estructura del AST:

- **total_nodes:** Número total de nodos en el AST.
- **max_depth:** Profundidad máxima del AST.

II. Declaraciones y Variables:

- **variables:** Número de variables locales.
- **constants:** Número de constantes declaradas.
- **variable_names:** Conjunto de nombres de variables y sus tipos.
- **number_of_tuples:** Número de variables de tipo tupla.
- **lists:** Número de listas declaradas.
- **dicts:** Número de diccionarios declarados.

Las variables y constantes almacenan valores que pueden cambiar o permanecer fijos durante la ejecución. Las variables revelan el comportamiento dinámico del código, mientras que las constantes indican valores fijos que definen parámetros o condiciones estables dentro del flujo de ejecución.

Además, la variedad y el tipo de estructuras de datos, como tuplas, listas y diccionarios, aportan información sobre el enfoque y la complejidad del código. Por ejemplo, el uso de estructuras de datos, como diccionarios anidados o listas de objetos, puede reflejar abstracción y modularidad en el diseño, mientras que estructuras más simples pueden indicar un código directo.

III. Declaraciones de Métodos y Clases:

- **methods:** Número de métodos declarados.
- **method_names:** Conjunto de nombres de métodos.
- **method_return_types:** Conjunto de tipos de retorno de métodos.
- **method_parameters:** Lista de parámetros de métodos.
- **classes:** Número de clases declaradas.
- **class_names:** Conjunto de nombres de clases.
- **abstract_classes:** Número de clases abstractas.
- **sealed_classes:** Número de clases selladas.
- **interfaces:** Número de interfaces declaradas.
- **interface_names:** Conjunto de nombres de interfaces.

La organización y modularidad del código se reflejan en la estructura y en los nombres de los métodos y clases. La interacción de los métodos con otros componentes evidencia el nivel de cohesión y diseño de la aplicación. Asimismo, las clases y sus tipos, como las abstractas o selladas, proporcionan información sobre la arquitectura y el enfoque de diseño del sistema.

IV. Estructuras de Control:

- **control_structures_if:** Número de sentencias if.
- **control_structures_switch:** Número de sentencias switch.
- **control_structures_for:** Número de bucles for.
- **control_structures_while:** Número de bucles while.
- **control_structures_dowhile:** Número de bucles do-while.
- **try_catch_blocks:** Número de bloques try-catch.

Las estructuras de control definen el flujo del programa y su lógica. Un mayor número de estructuras de control indica una lógica ramificada.

V. Modificadores y Accesibilidad:

- **access_modifiers_public:** Número de elementos públicos.
- **access_modifiers_private:** Número de elementos privados.
- **access_modifiers_protected:** Número de elementos protegidos.
- **access_modifiers_internal:** Número de elementos internos.
- **access_modifiers_static:** Número de elementos estáticos.
- **access_modifiers_protected_internal:** Número de elementos protegidos internos.
- **access_modifiers_private_protected:** Número de elementos privados protegidos.

Los modificadores de acceso proporcionan información sobre la encapsulación y visibilidad de los componentes del código. La prevalencia de ciertos modificadores puede indicar prácticas de diseño y seguridad en el código.

VI. Modificadores Específicos:

- **modifier_readonly:** Número de elementos readonly.
- **modifier_volatile:** Número de elementos volatile.
- **modifier_virtual:** Número de elementos virtual.
- **modifier_override:** Número de elementos override.
- **modifier_new:** Número de elementos new.

- **modifier_partial:** Número de elementos partial.
- **modifier_extern:** Número de elementos extern.
- **modifier_unsafe:** Número de elementos unsafe.
- **modifier_async:** Número de elementos async.

Al analizar modificadores como `readonly`, `volatile` o `async`, se obtiene información sobre cómo el código maneja la concurrencia, la inmutabilidad y la asincronía. Estos modificadores se emplean para identificar similitudes en la lógica y el flujo de ejecución. Modificadores como `virtual` y `override` indican una arquitectura orientada a objetos, lo que permite comparar el grado de extensibilidad y personalización en distintos fragmentos de código. Además, la presencia de `unsafe` y `extern` sugiere interacción con recursos de bajo nivel o externos, lo que proporciona información sobre las dependencias.

VII. Llamadas a Librerías y LINQ(Language Integrated Query):

- **library_call_console:** Número de llamadas a la librería Console.
- **library_call_math:** Número de llamadas a la librería Math.
- **linq_queries_select:** Número de consultas LINQ Select.
- **linq_queries_where:** Número de consultas LINQ Where.
- **linq_queries_orderBy:** Número de consultas LINQ OrderBy.
- **linq_queries_groupBy:** Número de consultas LINQ GroupBy.
- **linq_queries_join:** Número de consultas LINQ Join.
- **linq_queries_sum:** Número de consultas LINQ Sum.
- **linq_queries_count:** Número de consultas LINQ Count.

Las llamadas a librerías y las consultas LINQ reflejan el uso de funcionalidades estándar y la gestión de datos. Las primeras indican el acceso a herramientas optimizadas, lo que puede denotar la experiencia del programador en modularidad y adaptabilidad. Las consultas LINQ, por su parte, facilitan el acceso eficiente y la manipulación de colecciones de datos, lo que sugiere preferencia por una sintaxis declarativa y gestión avanzada de datos en .NET.

VIII. Otras Características:

- **number_of_lambdas:** Número de expresiones lambda.
- **number_of_getters:** Número de métodos get.
- **number_of_setters:** Número de métodos set.
- **number_of_namespaces:** Número de espacios de nombres.
- **enums:** Número de enumeraciones.
- **enum_names:** Conjunto de nombres de enumeraciones.
- **delegates:** Número de delegados.
- **delegate_names:** Conjunto de nombres de delegados.
- **node_count:** Conteo de nodos por tipo.

El número de expresiones lambda indica el uso de funciones anónimas, lo que sugiere una orientación hacia la programación funcional. La cantidad de métodos de acceso refleja el manejo de encapsulamiento y control de atributos. La presencia de espacios de nombres señala la estructura modular y la separación de responsabilidades.

La extracción de características mediante el uso de FeatureExtractor-Listener captura información del código fuente en C#, incluyendo su estructura, complejidad, patrones de diseño y prácticas de programación. Estas se procesan y se utilizan en el conjunto de entrenamiento, que será explicado en el próximo capítulo

Capítulo 3

Preparación del conjunto de entrenamiento (*dataset*)

En este capítulo se detalla la construcción de los vectores de características que sirven como entrada a la Red Neuronal que se propone y la construcción del conjunto de entrenamiento.

3.1. Word2Vec

En el contexto del análisis de similitud de código, los nombres de variables, métodos y otros identificadores en el código fuente proporcionan información semántica sobre la funcionalidad y el propósito de diferentes partes del código. Sin embargo, los identificadores en el código no están estructurados de manera que las máquinas puedan comprender sus relaciones semánticas, para lograr esto se utilizó Word2Vec [43]. Por ejemplo, los identificadores que se usan de manera similar en diferentes contextos tendrán *embeddings* similares, esto es una técnica de procesamiento de lenguaje natural que convierte el lenguaje natural en vectores numéricos. Estos vectores son una representación del significado subyacente de las palabras. El proceso involucra los siguientes pasos:

- I. *Extracción de Identificadores*: se extraen todos los nombres de variables, métodos, clases, interfaces, enumeraciones y delegados del código fuente utilizando la clase `FeatureExtractorListener`.
- II. *Entrenamiento de Word2Vec*: se utiliza un corpus de identificadores

que se extrajeron de proyectos de C# para entrenar el modelo Word2Vec. El modelo aprende las relaciones semánticas [53] entre los diferentes identificadores en el contexto del código.

- III. *Conversión a embeddings* [43]: se halla el promedio entre los vectores que corresponden a cada *feature* para asegurar que todos los vectores tengan la misma dimensión. Así, se captura la semántica y el contexto de los identificadores en el código.

Estos vectores forman parte de los vectores de características que se usan como entradas del modelo que se propone en este trabajo.

3.2. Vectores de características

Para preparar el conjunto de entrenamiento, los *features* que se extraen del AST se transforman en vectores de características numéricas (figura 3.1). Esto permite que un modelo de aprendizaje los reciba como entrada.

Para transformar en valores numéricos los nombres de variables, nombres de clases, nombres de interfaces y otros identificadores de tipo *string* se utiliza embeddings con el modelo Word2Vec (como se explicó en el capítulo anterior). Esto permite representar características no numéricas como vectores (y_i en la figura 3.1).

Estos embeddings se combinan con otras características numéricas del código como: cantidad de variables, cantidad de métodos, total de nodos y profundidad del AST (x_i en la figura 3.1)

La unión de los x_i y los y_i forma un vector de características completo. Este vector proporciona una descripción multidimensional del código y captura tanto la estructura como la semántica en una única representación.

3.3. Construcción del dataset

El *dataset* de códigos en C# está compuesto por dos tipos de pares de archivos:

- Aquellos que mantienen la misma funcionalidad pero presentan variaciones en la estructura y nomenclatura. Cada par está compuesto por 2 archivos que no son copias o un archivo original y su respectiva copia, con cambios mínimos y modificaciones que buscan simular

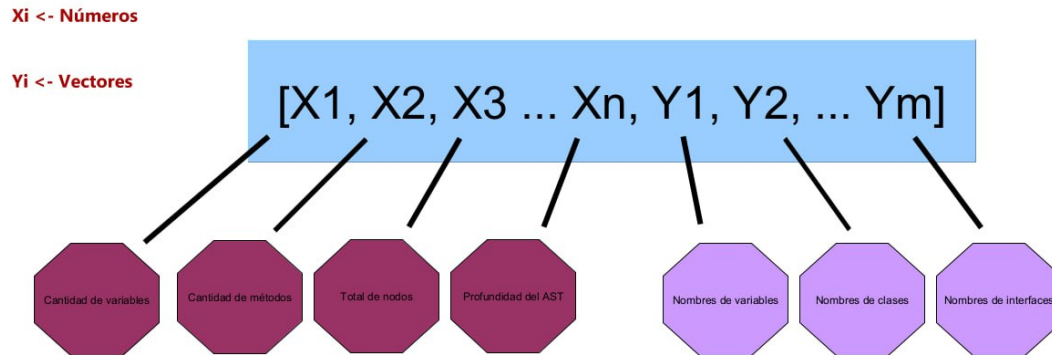


Figura 3.1: Ejemplo de vector de características

escenarios típicos en los que el código mantiene su esencia lógica pero experimenta alteraciones en la sintaxis y organización.

- Aquellos que se clasifican como diferentes.

Se eligieron las siguientes inteligencias artificiales generativas para generar parejas de códigos: 300 parejas generadas por GPT-4o, 300 por Copilot, 150 por Perplexity y 90 por Phind. Se usó un total de 1000 parejas para el entrenamiento, de ellas, 500 se clasificaron como similares (un código se generó a partir del otro) y 500 se clasificaron como distintas.

Se crearon múltiples fragmentos de código en C# que abarcan una variedad de funciones comunes, como cálculos geométricos, operaciones matemáticas, manipulación de cadenas de texto y validaciones de números.

Además, se eligió un conjunto de 100 parejas de proyectos de programación en C#. De esas, 30 parejas se generaron como copias manuales y 70 se crearon como cruces de códigos disímiles.

Para las 30 parejas similares, para cada archivo o proyecto original, se generó una copia modificada siguiendo las siguientes estrategias de variación:

- *Renombramiento de Clases, Métodos y Variables*: los nombres de clases, métodos y variables fueron modificados en las copias para simular

diferencias en nomenclatura sin cambiar el objetivo funcional del código.

- *Modificación de Estructura de Código:* se alteraron estructuras sintácticas, como el tipo de bucles (de `for` a `while`) o la reestructuración de condiciones lógicas como por ejemplo: dividir un proceso complejo en pasos más pequeños de manera diferente o reordenar las instrucciones sin cambiar el resultado esperado. Estas modificaciones permiten que la copia mantenga la misma funcionalidad que el original, pero con una apariencia diferente en términos de código.
- *Ajustes en la Organización:* Algunos fragmentos se reorganizaron en cuanto al orden de ejecución o el uso de estructuras de control alternativas, manteniendo los resultados finales consistentes con el código original.

Como resultado, se obtuvo un conjunto de datos que incluye código sintético (1000 parejas) y no sintético (100 parejas) ya que las copias que se generaron a mano se pueden considerar plagios reales.

Capítulo 4

Redes Neuronales Siamesas para la Similitud de Código

Las redes neuronales siamesas se emplearon para analizar la estructura y el comportamiento lógico del código. Este enfoque facilita la identificación de patrones y las relaciones textuales y sintácticas. En este capítulo se detallan los componentes de esta arquitectura, la implementación del modelo y el proceso que se usó para el entrenamiento y predicción.

4.1. Red Neuronal Siamesa

Una red neuronal siamesa (figura 4.1) consta de dos subredes idénticas que procesan dos entradas en paralelo y producen representaciones vectoriales latentes. Una representación vectorial latente es una forma de representar datos, como palabras, fragmentos de texto o código, en un espacio vectorial de alta dimensión. Estas capturan la información subyacente y las relaciones estructurales de los datos. La arquitectura básica de una red neuronal siamesa es la siguiente:

- **Entrada:** Dos vectores x_1 y x_2 de tamaño n .
- **Subred Base:** Ambas entradas se procesan mediante una red compartida (mismas capas y pesos para ambas entradas) f_θ :

$$d_1 = f_\theta(x_1), \quad d_2 = f_\theta(x_2).$$

que producen representaciones latentes.

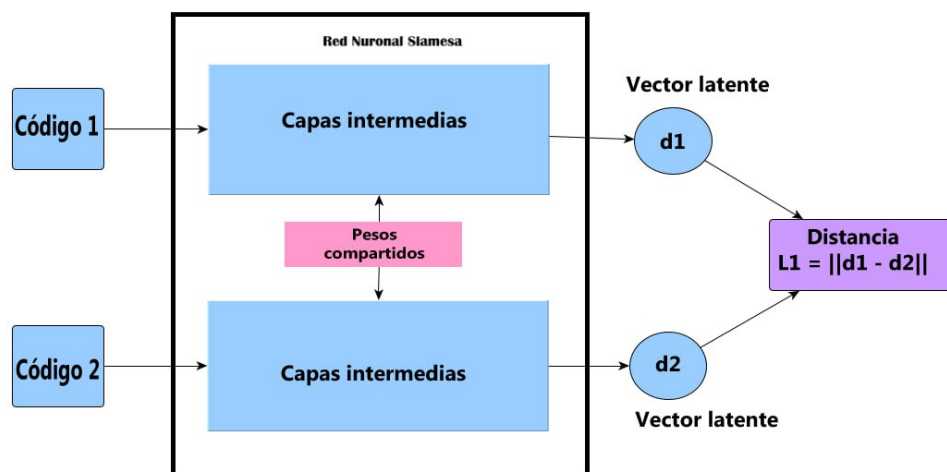


Figura 4.1: La imagen muestra la estructura general de una Red Neuronal Siamesa

- **Métrica de Distancia:** La similitud entre las representaciones latentes se mide mediante una distancia como se muestra en la figura 4.1

$$d(d_1, d_2) = ||d_1 - d_2||_p,$$

donde $p = 1$ para la distancia $L1$ [12], que se utiliza en este modelo.

- **Clasificación Binaria:** La distancia se pasa por una capa de activación sigmoideal [25] para obtener una probabilidad:

$$\hat{y} = \sigma(W \cdot d(d_1, d_2) + b).$$

4.2. Implementación del Modelo

Esta sección describe cómo se implementa un modelo de red siamesa utilizando TensorFlow/Keras [1]. La implementación incluye tres componentes: la subred base, la arquitectura siamesa y los mecanismos de optimización.

4.2.1. Subred Base

La subred base transforma las entradas $x \in \mathbb{R}^{65}$ (la dimensión se corresponde con el número de *features* que se extraen del AST) en representaciones vectoriales como se explicó en la sección 3.2. Este proceso ayuda al modelo a identificar patrones y eliminar información redundante.

Las capas de la subred base son las siguientes:

- I. **Capa densa (*Dense*):** cada capa densa aplica una transformación lineal seguida de una función de activación no lineal **ReLU** (Rectified Linear Unit) [23]. La fórmula matemática para una capa es:

$$h^{(l)} = \text{ReLU}(W^{(l)} \cdot h^{(l-1)} + b^{(l)}),$$

donde:

- $W^{(l)}$ es la matriz de pesos.
 - $b^{(l)}$ es el vector de sesgos.
 - $h^{(l)}$ es la salida de la capa l .
- II. **Regularización con *Dropout*:** Dropout apaga aleatoriamente una proporción $p = 0,5$ de las neuronas en cada capa durante el entrenamiento. Esto previene el sobreajuste del modelo [55].

Arquitectura detallada de la subred:

- Entrada $x \in \mathbb{R}^{65}$, donde cada dimensión es una característica del conjunto de datos.
- Tres capas densas con tamaños decrecientes (512, 256, y 128 neuronas) y activaciones **ReLU**.
- Salida $h(x) \in \mathbb{R}^{128}$, un vector de representación latente que condensa la información relevante de las entradas

Esta arquitectura se utiliza en las dos ramas del modelo siamés. El orden de en que se aplican las capas en cada rama se muestra en la figura 4.2.

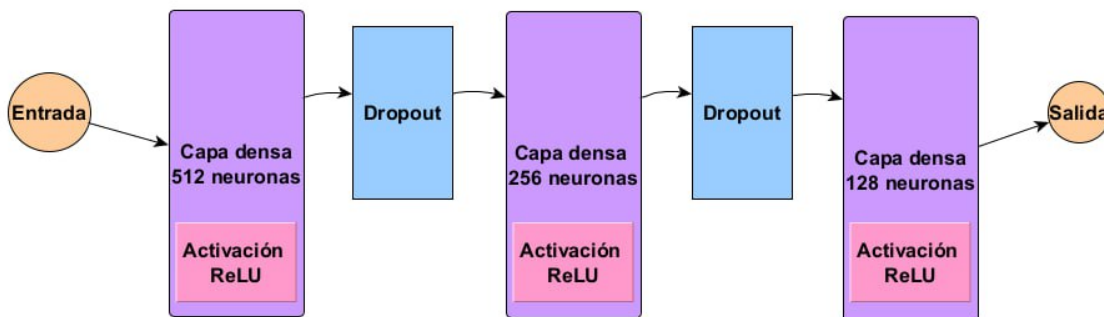


Figura 4.2: Capas de la subred base

4.2.2. Construcción del Modelo Siamés

El modelo siamés utiliza dos copias idénticas de la subred base para procesar dos entradas, compararlas y calcular una probabilidad de similitud.

- I. **Entradas:** Dos vectores $x_1, x_2 \in \mathbb{R}^{65}$ que representan las instancias a comparar.
- II. **Representaciones Latentes:** Cada entrada se transforma en una representación latente compacta mediante la subred base:

$$d_1 = f_{\theta}(x_1), \quad d_2 = f_{\theta}(x_2),$$

donde f_{θ} representa la subred base con parámetros compartidos θ .

- III. **Cálculo de la Distancia L1:** Para medir la similitud entre las representaciones latentes, se calcula la distancia L1:

$$d(d_1, d_2) = ||d_1 - d_2||.$$

Esto se implementa utilizando una capa **Lambda** en TensorFlow/Keras.

IV. **Clasificación:** La distancia calculada se pasa a una capa densa con una activación sigmoideal (**sigmoid**) que produce una probabilidad:

$$\hat{y} = \sigma(W \cdot d(d_1, d_2) + b),$$

donde:

- W son los pesos de la capa.
- b es el sesgo.
- $\sigma(z) = \frac{1}{1+e^{-z}}$ convierte z en un valor entre 0 y 1, que se interpreta como la probabilidad de similitud.

4.2.3. Función de Pérdida

En el contexto del aprendizaje automático, una **función de pérdida** (también llamada **función de coste**) es una medida de cuán bien o mal funciona un modelo de aprendizaje automático. En términos simples, evalúa la diferencia entre las predicciones que hace el modelo y los valores reales (etiquetas) que se observan en los datos de entrenamiento.

El objetivo del modelo durante el entrenamiento es minimizar la **función de pérdida**, lo que significa ajustar sus parámetros (como los pesos de una red neuronal) de manera que las predicciones del modelo se acerquen lo más posible a los valores reales.

En este trabajo se implementó una **función de pérdida** personalizada que introduce una penalización adicional a los falsos negativos. A diferencia de la función estándar *binary_crossentropy*, que considera una penalización uniforme para errores. Para la red que se propone, un falso positivo es una pareja de códigos que se detectan falsamente como copias, mientras que un falso negativo es una pareja de poyectos similares que no se puede detectar. El software que este trabajo propone tiene el objetivo de capturar posibles códigos similares, para que luego un experto humano (profesor) los clasifique con exactitud. En este contexto resulta preferible devolver falsos plagios antes de fallar en detectar plagios reales.

La **función de pérdida** personalizada se define como:

$$\text{Loss} = \text{binary_crossentropy}(y_{\text{true}}, y_{\text{pred}}) - \alpha \cdot y_{\text{true}} \cdot \log(1 - y_{\text{pred}} + \epsilon)$$

Donde:

- y_{true} : Etiqueta real (0 o 1).
- y_{pred} : Predicción del modelo (probabilidad entre 0 y 1).
- α : Factor de penalización para falsos negativos.
- ϵ : Valor pequeño para evitar indeterminaciones en el cálculo del logaritmo.

4.2.4. Optimización

El modelo utiliza el optimizador **Adam** (Adaptive Moment Estimation) para ajustar los pesos θ [24]. Adam combina las ventajas de:

- **Momentum**: acelera el entrenamiento en direcciones consistentes [52].
- **Tasas de aprendizaje adaptativas**: ajusta la tasa de aprendizaje individual para cada parámetro [20].

Adam recibe los siguientes parámetros de entrada:

- **Gradiente de la función de pérdida**: g_t (gradiente en el paso t)
- **Tasa de aprendizaje**: α (por defecto 0,001)
- **Tasas de decaimiento para los momentos**:
 - β_1 (para el término de Momentum, típicamente 0,9)
 - β_2 (para el término de RMSProp, típicamente 0,999)
- **Pequeña constante** ϵ para evitar divisiones por cero (por defecto 10^{-8})

- **Valores iniciales de los parámetros:** θ_0 (pesos a optimizar)
- **Paso de tiempo:** t , inicializado en 0.

En cada iteración t , Adam actualiza los parámetros siguiendo los siguientes pasos:

I. Cálculo del Primer Momento (Término de Momentum)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (4.1)$$

Este término representa un **promedio móvil exponencial de los gradientes anteriores**, y suaviza las actualizaciones.

II. Cálculo del Segundo Momento (Término de RMSProp)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4.2)$$

Este término mantiene un **promedio móvil exponencial de los cuadrados de los gradientes**.

III. Corrección del Sesgo: dado que m_t y v_t están inicialmente en cero, se debe corregir este sesgo mediante:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (4.3)$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (4.4)$$

IV. Actualización de los Parámetros: usando los momentos corregidos, se actualizan los parámetros θ_t de la siguiente manera:

$$\theta_t = \theta_{t-1} - \frac{\alpha \hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \quad (4.5)$$

- La actualización **ajusta el tamaño del paso** usando la raíz cuadrada del segundo momento ($\hat{v}_t$).
- La tasa de aprendizaje α se ajusta automáticamente según la varianza de los gradientes.

V. Incremento del Paso de Tiempo

$$t = t + 1 \quad (4.6)$$

El algoritmo devuelve los **parámetros actualizados** θ_t , moviéndolos en la dirección que minimiza la función de pérdida.

4.3. Proceso de Entrenamiento y Predicción

En esta sección se explica cómo se lleva a cabo el proceso de entrenamiento y predicción en el modelo siamés y cómo se predice la similitud entre las entradas.

4.3.1. Entrenamiento

Se entrena el modelo siames utilizando un conjunto de tuplas de datos (x_1, x_2, y) , donde x_1 y x_2 son dos instancias que se comparan entre sí, y y es la etiqueta binaria que indica si las instancias son similares ($y = 1$) o disímiles ($y = 0$).

Durante el entrenamiento, el modelo ajusta los pesos para minimizar la **función de pérdida** sobre el conjunto de entrenamiento. Se optimiza el modelo con el algoritmo de **Adam**, como se mencionó en la sección 4.2.4.

El modelo procesa cada par de entradas x_1, x_2 a través de la subred base para calcular sus representaciones latentes d_1 y d_2 . Luego, calcula la distancia entre estas representaciones y utiliza esta información para una predicción $\hat{y}$, que se compara con la etiqueta y en la función de pérdida. A medida que el modelo se entrena, ajusta sus pesos para minimizar la diferencia entre las predicciones $\hat{y}$ y las etiquetas y .

4.3.2. Predicción

Después de entrenar el modelo, el mismo se puede utilizar para hacer predicciones sobre nuevos pares de características $\mathbf{X}_1$ y $\mathbf{X}_2$. La

similitud entre los dos fragmentos de código se calcula pasando las características de cada fragmento a través del modelo y obteniendo un valor numérico de similitud.

El valor de la similitud, se obtiene de la siguiente forma:

$$\text{similarity} = \text{model.predict}([\mathbf{X}_1, \mathbf{X}_2]).$$

Este valor es un número entre 0 y 1, que indica cuán similares son los dos fragmentos de código. Un valor cercano a 1 indica que los fragmentos son similares, mientras que un valor cercano a 0 indica que son disímiles. La similitud se basa en la activación de la red, que convierte la distancia entre las representaciones latentes de los fragmentos en un valor de probabilidad.

Capítulo 5

Experimentos y resultados

En este capítulo se presentan los resultados que se obtuvieron durante la experimentación, así como los factores que influyeron en el rendimiento del modelo propuesto.

5.1. Evaluación del Modelo

Para evaluar el modelo se utiliza un conjunto de datos de prueba compuesto por pares de instancias etiquetados. El objetivo de esta evaluación es medir el desempeño del modelo en términos de la precisión (*precision*), exhaustividad (*recall*), y exactitud (*accuracy*). Además, se analizaron los errores para identificar áreas de mejora.

El modelo se evaluó de la siguiente forma:

- I. **Cálculo de pérdida y precisión:** Se calcularon la pérdida y la accuracy promedio en el conjunto de datos de prueba.

```
loss, accuracy = self.model.evaluate([data_a,data_b],labels,verbose = 1)
```

- II. **Predicción con umbral ajustado:** Se generaron predicciones con un umbral definido para clasificar las instancias. En este caso, el umbral fue (0,10), que se seleccionó para priorizar los

verdaderos positivos.

```
threshold = 0,10
predictions = (self.model.predict([data_a,data_b]) > threshold)
astype(int).flatten()
```

- III. **Análisis de la matriz de confusión:** Se analizó la matriz de confusión para identificar los errores del modelo y categorizar las instancias en verdaderos positivos (*TP*), falsos positivos (*FP*), verdaderos negativos (*TN*), y falsos negativos (*FN*).

Tabla 5.1 Métricas de rendimiento del modelo

Métrica	Valor
Verdaderos Positivos (TP)	98
Falsos Positivos (FP)	21
Verdaderos Negativos (TN)	75
Falsos Negativos (FN)	2

- IV. **Identificación de errores:** Se registraron los pares de datos que resultaron en falsos negativos y falsos positivos para analizar sus características. Esto permitió identificar posibles patrones en los errores.

5.2. Experimentos

La experimentación tuvo como objetivo principal reducir los falsos negativos (como se explicó en la sección 4.2.3) y, con ello, aumentar el recall del modelo. Este enfoque responde a la necesidad de priorizar la sensibilidad del modelo en detrimento de la precisión, por la importancia de capturar correctamente todos los casos positivos, proyectos que son fraude, incluso a costa de generar un mayor número de falsos positivos.

Esto resultó en una disminución del accuracy en comparación con la función de pérdida `binary_crossentropy`.

Para evaluar el rendimiento del modelo, se utilizaron las métricas estándar descritas a continuación:

- **Accuracy:** Mide la proporción de predicciones correctas sobre el total de predicciones. Se define como:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

donde TP representa los verdaderos positivos, TN los verdaderos negativos, FP los falsos positivos y FN los falsos negativos.

- **Precision:** Evalúa la proporción de verdaderos positivos entre todas las predicciones clasificadas como positivas por el modelo:

$$Precision = \frac{TP}{TP + FP}$$

En esta fórmula, el aumento de los falsos positivos (FP) incrementa el denominador, lo que resulta en una disminución de la precisión. Esto refleja que el modelo clasifica incorrectamente más ejemplos como positivos, lo que reduce la proporción de verdaderos positivos respecto al total de predicciones positivas.

- **Recall:** Mide la proporción de verdaderos positivos capturados por el modelo respecto al total de ejemplos positivos reales:

$$Recall = \frac{TP}{TP + FN}$$

En esta ecuación, el aumento de los verdaderos positivos (TP) incrementa el numerador, lo que lleva a un mayor valor de *recall*. Asimismo, la disminución de los falsos negativos (FN) reduce el denominador, lo que también incrementa esta métrica. Por tanto, el *recall* es directamente proporcional a los verdaderos positivos y de forma inversa proporcional a los falsos negativos.

El diseño de una función de pérdida personalizada que enfatiza la penalización de los falsos negativos permitió optimizar el *recall*, aunque a expensas de reducir la *precision* y la *accuracy*. Este enfoque prioriza minimizar los errores en la identificación de ejemplos positivos, incluso cuando ello conlleva un aumento en los errores al clasificar

negativos como positivos. Si bien esta estrategia no es adecuada en contextos donde la precisión es prioritaria, es efectiva en problemas donde los falsos negativos representan un riesgo para el sistema.

A continuación se presentan los resultados de los casos de pruebas donde se utilizaron las dos funciones de pérdida implementadas, se obtuvo que la función de pérdida personalizada logró mejores resultados en términos de `recall`, mientras que `binary_crossentropy` mantuvo un `recall` ligeramente inferior, pero consiguió un `accuracy` considerablemente mayor.

Tabla 5.2 Resultados

Función de pérdida	acc	loss	precision	recall
<code>binary_crossentropy</code>	0.9082	0.4948	0.9192	0.9100
<code>assymetric_loss</code>	0.8469	1.1607	0.8235	0.9800

El `recall` cercano a 1.0 de la función de pérdida personalizada resultó en un modelo capaz de detectar una inmensa mayoría de los proyectos similares al comparar dos a dos un conjunto de códigos en lenguaje C#. Además, con `binary_crossentropy` se entrenó otro modelo que captura una gran cantidad de plagios, y da, al mismo tiempo, pocos falsos positivos. Este es el principal aporte del presente trabajo y el siguiente capítulo sirve como validación para este resultado.

5.3. Validación de resultados

Para validar los resultados de la tesis se tomaron 4 orientaciones de proyectos reales que se impartieron en la asignatura de Programación de MATCOM primer año. Para clasificar se usó el modelo que se entrenó con la función de pérdida personalizada que se presentó en la sección 4.2.3. La tabla 5.3 muestra los resultados de **MoogLe** [39], la tabla 5.4 muestra los resultados de **Dominó** [37], la tabla 5.5 muestra los resultados de **Hulk** [38] y la tabla 5.6 muestra los resultados de **Walle** [40]. En cada caso se tomó un conjunto de 10 proyectos implementados por estudiantes y a un subconjunto aleatorio de 5 proyectos (por cada grupo de 10 se escogieron 5), se le generó una

copia, simulando lo que haría un estudiante para cometer fraude. Por ejemplo:

- Cambios en nombres de variables, métodos y clases.
- Cambios estructurales (Como orden de declaraciones de variables) que no afectan la lógica del código.
- Intercambios de tipos similares (Como intercambios de **log** por **int** y **double** por **float**).
- Cambios sintácticos que no afectan la lógica del código (Como intercambios de condicionales **Switch** por conjuntos de condicionales **if**)

En cada tipo de proyecto (Moogles, Dominó, Hulk y WallE) se probaron, por tanto, 5 parejas de proyectos que se clasifican como **plagio** y $\binom{15}{2} - 5 = 100$ (Todas las parejas dos a dos menos las 5 parejas de proyectos clonadas) que no se clasifican como **plagio**.

Cada una de las 4 tablas (5.3, 5.4, 5.5 y 5.6 presenta 4 columnas:

- **Parejas plagio:** Pares de proyectos copia que se generaron.
- **Parejas no plagio:** Proyectos no similares que se emparejaron dos a dos.
- **Plagios detectados:** Parejas de proyectos copia que se generaron y fueron detectados por el modelo.
- **Plagios no detectados:** Parejas de proyectos copia que se generaron y el algoritmo no fue capaz de detectar.
- **Falsos plagios:** Parejas de proyectos no similares que el algoritmo detectó erróneamente como copias.

Los resultados que se presentaron en este capítulo muestran que el modelo es capaz de identificar correctamente los proyectos fraudulentos en una gran mayoría de los casos. Sin embargo, se capturaron también erróneamente varios proyectos no fraudulentos que deben ser filtrados por expertos (profesores).

Tabla 5.3 Resultados de la detección de similitud de Moogle

Parejas plagio	Parejas NO plagio	Plagios detectados	Plagios no detectados	Falsos plagios
5	100	5	0	18

Tabla 5.4 Resultados de la detección de similitud de Dominó

Parejas plagio	Parejas NO plagio	Plagios detectados	Plagios no detectados	Falsos plagios
5	100	5	0	20

Tabla 5.5 Resultados de la detección de similitud de Hulk

Parejas plagio	Parejas NO plagio	Plagios detectados	Plagios no detectados	Falsos plagios
5	100	5	0	22

Tabla 5.6 Resultados de la detección de similitud de Walle

Parejas plagio	Parejas NO plagio	Plagios detectados	Plagios no detectados	Falsos plagios
5	100	5	0	25

Capítulo 6

Conclusiones

En el presente Trabajo de Diploma, se desarrolló una herramienta computacional capaz de detectar, dado un conjunto de códigos de C#, parejas de códigos similares. A partir de este resultado, los profesores de la asignatura de Programación, u otras donde los estudiantes utilicen este tipo de código, pueden usar dicha herramienta para detectar posibles fraudes en sus trabajos evaluativos. Los principales aportes de esta investigación son los siguientes:

- I. Un software de detección de similitud de código en C# que extrae características de ASTs y las utiliza como entrada para un modelo de Aprendizaje Automático.
- II. Una herramienta práctica y de código libre que se puede usar con fines didácticos con experimentos que validan su utilidad.
- III. Se actualizó la gramática de C# de ANTLR, de la versión 6.0 a la 12.0.

Recomendaciones

Se recomienda dar continuidad al presente trabajo de investigación obteniendo los siguientes resultados:

- Entrenar el modelo para detectar similitudes de código en otros lenguajes de programación como Python y Java.
- Crear un repositorio de GitHub de código abierto para centralizar estudios posteriores (entrenamiento con nuevos *datasets* para diferentes lenguajes de programación)
- Dar utilizad práctica en el ámbito educativo al *software* propuesto, usándolo para la detección de posibles fraudes en la asignatura de Programación en el 1er. año de la carrera de Licenciatura en Ciencias de la Computación en la facultad de Matemática y Computación de la Universidad de la Habana.

Anexos

Cambios de la gramática de C#.

C# 7.0

Variables out

Problema: El uso de variables out requería una declaración previa. Desde C# 7.0, se permite declarar variables directamente en la declaración del método.

Solución: Se modificó la regla `argument` para incluir una declaración de variable opcional antes del modificador out.

Además, se ajustó la regla `local_variable_declaration` para permitir el uso de out como modificador de variable.

Tuplas y Deconstrucción

Problema: Las tuplas no eran un tipo nativo y requerían una biblioteca externa. Desde C# 7.0, se introdujeron tuplas como un tipo nativo, permitiendo la deconstrucción de sus elementos.

Solución: Se agregó una nueva regla `tuple_type` para definir tipos de tuplas. Se modificó la regla `type` para incluir `tuple_type`. Además, se añadió una regla `tuple_literal` para la creación de tuplas y se actualizó la regla `assignment` para soportar la deconstrucción de tuplas.

Coincidencia de Patrones

Problema: No había soporte para coincidencia de patrones en C#. Desde C# 7.0, se introdujo la coincidencia de patrones que permite simplificar el código al verificar tipos y valores.

Solución: Se introdujo una nueva regla `pattern_matching_expression` que incluye subreglas para diferentes tipos de patrones (constante, tipo, var). Se actualizaron las reglas `is_expression` y `switch_section` para incorporar la nueva sintaxis de coincidencia de patrones.

Funciones Locales

Problema: Las funciones debían estar definidas a nivel de clase. Desde C# 7.0, se introdujeron funciones locales que permiten definir funciones dentro de métodos.

Solución: Se creó una nueva regla `local_function_declaration` similar a `method_declaration`, pero permitida dentro de bloques de código. Se actualizó la regla `statement` para incluir `local_function_declaration` como una opción válida.

Miembros Expresados con Cuerpo de Expresión Expandido

Problema: Los miembros debían tener un cuerpo completo. Desde C# 7.0, se introdujo la posibilidad de usar cuerpos de expresión para simplificar la definición de miembros.

Solución: Se modificaron las reglas `method_declaration`, `property_declaration`, `operator_declaration`, `constructor_declaration`, `destructor_declaration`, y `indexer_declaration` para permitir el uso de `=>` seguido de una expresión como alternativa al cuerpo del bloque tradicional.

Locales ref

Problema: No era posible utilizar locales `ref` que permiten referenciar variables directamente en lugar de copiar su valor. Desde C# 7.0,

se introdujeron los locales `ref`.

Solución: Se actualizó la regla `local_variable_declaration` para incluir el modificador `ref` opcional. Se agregó una nueva regla `ref_expression` para manejar expresiones que devuelven referencias.

Retornos `ref`

Problema: Los métodos no podían devolver referencias directamente. Desde C# 7.0, se introdujo el soporte para retornos `ref`, permitiendo que un método devuelva una referencia a un valor existente.

Solución: Se modificó la regla `method_declaration` para permitir el modificador `ref` antes del tipo de retorno. Se actualizó la regla `return_statement` para incluir la opción de devolver una referencia usando `ref`.

Discards

Problema: No había una forma explícita de ignorar valores en asignaciones o expresiones. Desde C# 7.0, se introdujo el uso de discards (`_`) que permite omitir valores que no son necesarios.

Solución: Se agregó una nueva regla `discard` que reconoce el símbolo `_`. Se actualizaron las reglas relevantes como `variable_declaration`, `foreach_statement`, y `out_variable_declaration` para permitir el uso de `discard` en lugar de identificadores.

Literales Binarios y Separadores de Dígitos

Problema: No había soporte para literales binarios ni separadores de dígitos en números. Desde C# 7.0, se introdujeron literales binarios y la capacidad de usar guiones bajos como separadores en literales numéricos.

Solución: Se modificó la regla `integer_literal` para incluir el prefijo `0b` o `0B` para literales binarios. Se actualizaron las reglas de literales numéricos para permitir guiones bajos (`_`) entre dígitos en cualquier posición excepto al principio o final del literal.

Expresiones `throw`

Problema: Las expresiones `throw` solo podían usarse en instrucciones completas. Desde C# 7.0, se introdujeron expresiones `throw`, permitiendo lanzar excepciones como parte de expresiones más complejas.

Solución: Se creó una nueva regla `throw_expression` que permite el uso de `throw` seguido de una expresión. Se actualizaron las reglas relevantes como `conditional_expression` y `null_coalescing_expression` para incluir `throw_expression` como una opción válida.

C# 7.1

Método `Main` Asíncrono

Problema: El método de entrada `Main` no podía ser asíncrono. Esto limitaba el uso de operaciones asíncronas en la inicialización de aplicaciones. Desde C# 7.1, se permite que el método `Main` tenga el modificador `async`.

Solución: Se ajustaron las reglas gramaticales para permitir el modificador `async` en el método de entrada `Main`.

Expresiones Literales por Defecto (`default`)

Problema: En versiones anteriores, era necesario especificar explícitamente el tipo al usar valores predeterminados. Desde C# 7.1, se introdujeron las expresiones literales `default`, que infieren automáticamente el tipo cuando es posible.

Solución: Se modificó la gramática para permitir expresiones literales `default` en contextos donde el tipo puede inferirse.

Inferencia de Nombres de Elementos en Tuplas

Problema: En versiones anteriores, era necesario nombrar explícitamente los elementos de una tupla al inicializarla. Desde C# 7.1, se permite inferir los nombres de los elementos a partir de las variables utilizadas en la inicialización.

Solución: Se ajustaron las reglas gramaticales para inferir automáticamente los nombres de los elementos en inicializaciones de tuplas.

Coincidencia de Patrones en Parámetros Genéricos

Problema: No era posible realizar coincidencia de patrones en variables cuyo tipo era un parámetro genérico. Desde C# 7.1, se permite realizar coincidencias de patrones en este caso.

Solución: Se expandieron las reglas para permitir coincidencia de patrones en variables con tipos genéricos.

C# 7.2

Inicializadores en Arreglos Stackalloc

Problema: No era posible inicializar directamente los arreglos creados con `stackalloc`. Desde C# 7.2, se permite usar inicializadores en arreglos `stackalloc`.

Solución: Se ajustó la gramática para permitir inicializadores en arreglos creados con `stackalloc`.

Uso de Sentencias Fijas con Tipos que Soportan Patrones

Problema: Las sentencias `fixed` solo podían usarse con tipos específicos como arreglos o cadenas. Desde C# 7.2, se permite usar sentencias `fixed` con cualquier tipo que soporte un patrón.

Solución: Se modificó la regla `fixed_statement` para permitir el uso de cualquier tipo que soporte patrones.

Acceso a Campos Fijos Sin Fijación

Problema: Para acceder a campos fijos era necesario fijar la dirección de memoria explícitamente. Desde C# 7.2, se permite acceder a campos fijos sin necesidad de fijarlos.

Solución: Se ajustaron las reglas gramaticales para permitir el acceso a campos fijos sin fijación explícita.

Reasignación de Variables Locales Referenciadas

Problema: No era posible reasignar variables locales referenciadas (ref). Desde C# 7.2, se permite reasignar estas variables.

Solución: Se modificaron las reglas para permitir la reasignación de variables locales referenciadas.

Declaración de Tipos Struct Solo de Lectura (readonly struct)

Problema: No había una forma explícita de declarar estructuras inmutables. Desde C# 7.2, se introdujo el modificador `readonly struct` para indicar que una estructura es inmutable y debe pasarse como un parámetro `in`.

Solución: Se agregó soporte para el modificador `readonly` en la declaración de estructuras.

Modificador `in` en Parámetros

Problema: No era posible pasar argumentos por referencia sin permitir su modificación. Desde C# 7.2, se introdujo el modificador `in`, que permite pasar argumentos por referencia sin permitir modificaciones.

Solución: Se ajustó la regla `parameter_modifier` para incluir el modificador `in`.

Modificador `ref readonly` en Retornos de Métodos

Problema: No había una forma explícita de devolver referencias inmutables desde métodos. Desde C# 7.2, se introdujo el modificador `ref readonly`, que permite devolver referencias inmutables.

Solución: Se agregó soporte para el modificador `ref readonly` en las reglas de retorno de métodos.

Declaración de Tipos Struct Referenciados (`ref struct`)

Problema: No había una forma explícita de declarar estructuras que accedan directamente a memoria administrada y deban ser asignadas en el stack. Desde C# 7.2, se introdujo el modificador `ref struct`.

Solución: Se agregó soporte para el modificador `ref struct` en la declaración de estructuras.

Argumentos Nombrados No Finales

Problema: En versiones anteriores, los argumentos nombrados debían colocarse al final de la lista de argumentos. Desde C# 7.2, se permite mezclar argumentos posicionales y nombrados siempre que los posicionales estén primero.

Solución: Se ajustaron las reglas gramaticales para permitir argumentos nombrados no finales.

Guiones Bajos Iniciales en Literales Numéricos

Problema: Los literales numéricos no podían comenzar con guiones bajos (`_`). Desde C# 7.2, se permite usar guiones bajos iniciales antes de los dígitos impresos.

Solución: Se ajustó la gramática para permitir guiones bajos iniciales en literales numéricos.

Modificador `private protected`

Problema: No había un modificador que combinara las restricciones de acceso `private` y `protected`. Desde C# 7.2, se introdujo el modificador `private protected`, que permite acceso desde clases derivadas dentro del mismo ensamblado.

Solución: Se agregó soporte para el modificador `private protected` en las reglas de acceso.

Expresiones Condicionales por Referencia

Problema: En versiones anteriores, las expresiones condicionales (`?:`) no podían devolver referencias. Desde C# 7.2, se permite que el resultado sea una referencia.

Solución: Se ajustó la gramática para permitir expresiones condicionales por referencia.

C# 7.3

Acceso a Campos Fijos Sin Fijación

Problema: En versiones anteriores de C#, para acceder a campos fijos era necesario fijar la dirección de memoria. Desde C# 7.3, se permite acceder a campos fijos sin necesidad de fijarlos explícitamente.

Solución: Se ajustaron las reglas gramaticales para permitir el acceso a campos fijos sin fijación.

Reasignación de Variables Locales Referenciadas

Problema: No se podía reasignar variables locales referenciadas. Desde C# 7.3, se permite la reasignación de variables locales que están marcadas como `ref`.

Solución: Se modificaron las reglas para permitir la reasignación de variables locales referenciadas.

Inicializadores en Arreglos Stackalloc

Problema: No era posible utilizar inicializadores con arreglos creados con `stackalloc`. Desde C# 7.3, se permite usar inicializadores en arreglos `stackalloc`.

Solución: Se ajustaron las reglas gramaticales para permitir inicializadores en arreglos `stackalloc`.

Sentencias Fijas con Tipos que Soportan Patrones

Problema: Las sentencias fijas solo podían usarse con tipos específicos. Desde C# 7.3, se permiten sentencias fijas con cualquier tipo que soporte patrones.

Solución: Se modificó la regla `fixed_statement` para permitir el uso con cualquier tipo que soporte patrones.

Pruebas con Tipos Tupla

Problema: No se podía usar `==` y `!=` directamente con tipos tupla. Desde C# 7.3, se permite comparar tuplas usando estos operadores.

Solución: Se modificaron las reglas para permitir comparaciones directas entre tipos tupla.

Uso de Variables de Expresión en Más Ubicaciones

Problema: Las variables de expresión tenían un alcance limitado. Desde C# 7.3, se permite usar variables de expresión en más ubicaciones dentro del código.

Solución: Se ajustaron las reglas gramaticales para ampliar el uso de variables de expresión en diferentes contextos.

Resolución de Métodos Mejorada al Usar Argumentos Diferentes por in

Problema: La resolución de métodos al usar argumentos diferentes por in era confusa y poco clara. Desde C# 7.3, se mejoró este aspecto.

Solución: Se ajustaron las reglas gramaticales para mejorar la resolución de métodos al usar argumentos por in.

C# 8

Miembros Solo de Lectura readonly

Problema: No se podía definir miembros de solo lectura en las clases. Desde C# 8, se introdujeron los miembros solo de lectura, que permiten que las propiedades sean inmutables después de su inicialización.

Solución: Se modificó la regla `property_declaration` para permitir miembros solo de lectura `readonly`.

Métodos de Interfaz por Defecto

Problema: No era posible definir métodos en interfaces con una implementación por defecto. Desde C# 8, se introdujeron métodos de interfaz por defecto, permitiendo que las interfaces contengan métodos implementados.

Solución: Se ajustó la regla `interface_declaration` para permitir métodos de interfaz con implementación por defecto.

Mejoras en Coincidencia de Patrones

Problema: Las versiones anteriores de C# tenían limitaciones en los patrones disponibles. Desde C# 8, se introdujeron mejoras en la coincidencia de patrones, incluyendo expresiones `switch` y patrones de propiedades.

Solución: Se expandieron las reglas `pattern` para incluir expresiones `switch` y patrones mejorados.

Patrones de Propiedades

Problema: No había soporte para patrones que coincidieran con propiedades específicas. Desde C# 8, se introdujeron patrones de propiedades para facilitar la coincidencia con las propiedades de un objeto.

Solución: Se modificó la regla `pattern` para incluir patrones que coincidan con propiedades.

Patrones Tupla

Problema: No había soporte para coincidencias basadas en tuplas. Desde C# 8, se introdujeron patrones `tupla` para facilitar la coincidencia con estructuras de tuplas.

Solución: Se agregó soporte para patrones `tupla` en la regla `pattern`.

Patrones Posicionales

Problema: No había soporte para patrones que coincidieran con la posición de los elementos. Desde C# 8, se introdujeron patrones posicionales que permiten coincidir elementos en una posición específica.

Solución: Se agregó soporte para patrones posicionales en la regla `pattern`.

Declaraciones Using

Problema: Las declaraciones `using` debían estar al principio del archivo o dentro del bloque. Desde C# 8, se permite utilizar declaraciones `using` dentro del bloque de código.

Solución: Se modificó la regla `using_declaration` para permitir declaraciones `using` más flexibles.

Funciones Locales Estáticas

Problema: Las funciones locales no podían ser estáticas antes. Desde C# 8, se permite definir funciones locales como estáticas.

Solución: Se agregó soporte para funciones locales estáticas en la gramática.

Estructuras Ref Descartables

Problema: No había un manejo específico para estructuras ref descartables. Desde C# 8, se introdujo el soporte para estructuras ref descartables que permiten un manejo más eficiente de la memoria.

Solución: Se modificó la regla `struct_declaration` para permitir estructuras ref descartables.

Tipos Nullable Referencia

Problema: Todas las referencias eran consideradas no nulas. Desde C# 8, se introdujo el concepto de tipos nullable referencia que permite indicar explícitamente si una referencia puede ser nula.

Solución: Se ajustaron las reglas gramaticales para incluir el manejo de tipos nullable referencia.

Flujos Asincrónicos

Problema: No había un manejo eficiente para flujos asincrónicos. Desde C# 8, se introdujo el soporte para flujos asincrónicos que permite trabajar con secuencias de datos asincrónicas.

Solución: Se modificaron las reglas gramaticales para permitir flujos asincrónicos y su uso en bucles.

Índices y Rangos

Problema: No había una forma sencilla de trabajar con índices y rangos en colecciones. Desde C# 8, se introdujo el soporte para índices y rangos que facilita el acceso a elementos en colecciones.

Solución: Se agregó soporte para índices y rangos en la gramática.

Asignación Null-Coalescing

Problema: No había forma concisa de asignar valores cuando una variable es nula. Desde C# 8, se introdujo el operador null-coalescing assignment (`??=`).

Solución: Se ajustó la gramática para incluir el operador null-coalescing assignment.

Tipos Construidos No Administrados

Problema: No había un manejo específico para tipos contruidos no administrados. Desde C# 8, se introdujo el soporte para tipos contruidos no administrados que permiten trabajar con tipos sin administración del recolector de basura.

Solución: Se agregó soporte para tipos contruidos no administrados en la gramática.

Stackalloc en Expresiones Anidadas

Problema: No era posible usar `stackalloc` dentro de expresiones anidadas. Desde C# 8, se permite usar `stackalloc` dentro de expresiones anidadas.

Solución: Se ajustó la gramática para permitir el uso de `stackalloc` en expresiones anidadas.

Mejora en las Cadenas Interpoladas Verbatim

Problema: Las cadenas interpoladas verbatim tenían limitaciones en su formato. Desde C# 8, se mejoraron las cadenas interpoladas verbatim para permitir más flexibilidad al incluir expresiones complejas.

Solución: Se modificó la regla `interpolated_string_expression` para mejorar el manejo de cadenas interpoladas verbatim.

C# 9

Registros `record`

Problema: No existía un tipo de datos específico para registros. Sin embargo, desde C# 9, se introdujeron los registros, que son tipos de referencia inmutables que facilitan la creación de objetos con propiedades.

Solución: Se modificó la regla `class_declaration` para permitir la definición de `record`.

Setters Solo de Inicialización `init`

Problema: Todas las propiedades podían ser modificadas después de su inicialización. Desde C# 9, se introdujo la posibilidad de definir propiedades con setters solo de inicialización, permitiendo que las propiedades sean asignadas solo durante la creación del objeto.

Solución: Se ajustó la regla `property_declaration` para incluir el modificador `init`.

Declaraciones en Nivel Superior

Problema: El código debía estar contenido dentro de una clase o espacio de nombres. Desde C# 9, se permite escribir declaraciones en nivel superior, lo que simplifica el código.

Solución: Se modificó la gramática para permitir declaraciones en nivel superior sin necesidad de definir una clase.

Mejoras en Coincidencia de Patrones

Problema: Las versiones anteriores de C# tenían limitaciones en los patrones disponibles. Desde C# 9, se introdujeron mejoras en la coincidencia de patrones, incluyendo patrones relacionales y lógicos.

Solución: Se expandieron las reglas `pattern` para incluir patrones relacionales (`>`, `<`, `≤`, `≥`) y lógicos(`or`, `and`, `not`).

Enteros de Tamaño Nativo

Problema: No había un tipo específico para enteros que se adaptara al tamaño nativo del sistema. Desde C# 9, se introdujeron enteros de tamaño nativo (`nint` y `nuint`).

Solución: Se agregó soporte para los tipos `nint` y `nuint` en la regla `type_`.

Punteros a Funciones

Problema: No era posible trabajar con punteros a funciones directamente. Desde C# 9, se introdujo el soporte para punteros a funciones.

Solución: Se agregó soporte para punteros a funciones en las reglas correspondientes.

Inicializadores de Módulo

Problema: No había una forma específica de inicializar módulos. Desde C# 9, se introdujeron inicializadores de módulo.

Solución: Se agregó soporte para inicializadores de módulo [`ModuleInitializer`] en la gramática.

Nuevas Funciones para Métodos Parciales

Problema: Las funciones parciales tenían limitaciones en su uso. Desde C# 9, se introdujeron nuevas características para métodos parciales.

Solución: Se ajustaron las reglas relacionadas con métodos parciales para incluir nuevas características.

Expresiones Nuevas con Tipos Objetivo

Problema: No había una forma concisa de crear instancias utilizando tipos objetivo. Desde esta versión, se permite crear expresiones tipadas directamente.

Solución: Se ajustó la gramática para permitir expresiones tipadas al usar `new`.

Funciones Anónimas Estáticas

Problema: Las funciones anónimas no podían ser estáticas anteriormente. Sin embargo, desde C# 9, se permite definir funciones anónimas como estáticas.

Solución: Se agregó soporte para funciones anónimas estáticas en la gramática.

Expresiones Condicionales Tipadas por Objetivo

Problema: Las expresiones condicionales no podían ser tipadas por objetivo antes. Desde C# 9, se permite que las expresiones condicionales tengan un tipo objetivo inferido.

Solución: Se modificó la gramática para permitir expresiones condicionales tipadas por objetivo.

Tipos Retornados Covariantes

Problema: Los tipos retornados covariantes $c ? e_1 : e_2$ no eran compatibles. Sin embargo, desde C# 9, se introdujo el soporte para tipos retornados covariantes en métodos virtuales o sobrescritos.

Solución: Se ajustó la regla `method_declaration` para permitir tipos retornados covariantes.

Soporte para Extensiones GetEnumerator en Bucles foreach

Problema: El bucle `foreach` requería que las colecciones implementaran explícitamente `IEnumerable`. Con esta versión, se permite un soporte más flexible mediante extensiones `GetEnumerator`.

Solución: Se modificó la regla correspondiente al bucle `foreach` para incluir soporte para extensiones `GetEnumerator`.

Parámetros Descartados en Lambdas

Problema: Los parámetros descartados no eran compatibles en expresiones lambda. Sin embargo, desde C# 9, se permite usar parámetros descartados (`_`) en lambdas.

Solución: Se ajustó la regla `lambda_expression` para permitir parámetros descartados.

Atributos en Funciones Locales

Problema: Anteriormente no era posible aplicar atributos a funciones locales. Sin embargo, desde C# 9, se permite aplicar atributos a funciones locales definidas dentro de métodos.

Solución: Se modificó la regla correspondiente a los atributos para permitir su uso en funciones locales.

Uso del operador with

Problema: Antes de C# 9.0, no existía una forma concisa y directa de crear copias de objetos inmutables con algunas propiedades modificadas. Esto dificultaba la manipulación de objetos inmutables, ya que requería la creación manual de nuevos objetos con las propiedades deseadas.

Solución: Se modificó la regla `object_initializer` para permitir el uso del operador `with`, que permite a los desarrolladores crear un nuevo objeto basado en uno existente y modificar solo las propiedades especificadas. Esto se implementó mediante la adición de una nueva regla `with_expression` en la gramática, que permite la sintaxis `objeto with { propiedad = valor }`.

C# 10

Estructuras de Registro `record` `struct`

Problema: Las estructuras de registro no estaban disponibles. Sin embargo, desde C# 10, se introdujeron las estructuras de registro, que permiten definir estructuras con características de registro.

Solución: Se modificó la regla `struct_declaration` para permitir estructuras de registro.

Manejadores de Cadenas Interpoladas

Problema: No había una forma explícita de manejar las cadenas interpoladas. Sin embargo, desde C# 10, se introdujeron los manejadores de cadenas interpoladas para mejorar su manejo.

Solución: Se ajustó la regla `interpolated_string_expression` para reconocer correctamente las expresiones dentro de cadenas interpoladas.

Directivas using Globales

Problema: Las directivas `using` debían estar dentro de un espacio de nombres. Sin embargo, desde C# 10, se permiten directivas `using` globales que pueden aplicarse a todo el archivo.

Solución: Se modificó la regla `using_directive` para permitir directivas `using` globales.

Declaración de Espacios de Nombres `namespace` a Nivel de Archivo

Problema: Los espacios de nombres debían declararse utilizando llaves (`{}`). Sin embargo, desde C# 10, también es posible declararlos con punto y coma (`;`), permitiendo definiciones más concisas.

Solución: Se amplió la regla `namespace_declaration` para soportar ambas sintaxis.

Patrones de Propiedades Extendidos

Problema: Los patrones de propiedades no eran tan flexibles. Sin embargo, desde C# 10, se extendieron los patrones de propiedades para permitir más flexibilidad en la coincidencia de patrones.

Solución: Se modificó la regla `pattern` para soportar patrones de propiedades extendidos `property_pattern`.

Expresiones Lambda con Tipo Natural

Problema: Las expresiones lambda debían tener un tipo explícito. Sin embargo, desde C# 10, se permite que las expresiones lambda tengan un tipo natural, donde el compilador puede inferir el tipo delegado.

Solución: Se modificó la regla `lambda_expression` para permitir tipos naturales en expresiones lambda.

Expresiones Lambda con Tipo de Retorno Explícito

Problema: Las expresiones lambda no podían declarar un tipo de retorno explícito. Sin embargo, desde C# 10, se permite que las expresiones lambda declaren un tipo de retorno cuando el compilador no puede inferirlo.

Solución: Se modificó la regla `lambda_expression` para permitir tipos de retorno explícitos.

Atributos en Expresiones Lambda

Problema: Los atributos no podían aplicarse a expresiones lambda. Sin embargo, desde C# 10, se permite aplicar atributos a expresiones lambda.

Solución: Se modificó la regla `lambda_expression` para permitir atributos.

Inicialización de Cadenas Constantes con Interpolación

Problema: Las cadenas constantes no podían inicializarse con interpolación. Sin embargo, desde C# 10, se permite inicializar cadenas constantes utilizando interpolación si todos los marcadores son cadenas constantes.

Solución: Se ajustó la regla `constant_expression` para permitir interpolación en cadenas constantes.

Modificador `sealed` en `ToString` de Registros

Problema: No había una forma explícita de sellar el método `ToString` en tipos de registro. Sin embargo, desde C# 10, se permite agregar el modificador `sealed` cuando se sobrescribe `ToString` en un tipo de registro.

Solución: Se modificó la regla `method_declaration` para permitir el modificador `sealed` en `ToString`.

Advertencias para Asignación Definida y Análisis de Estado Nulo

Problema: Las advertencias para asignación definida y análisis de estado nulo no eran tan precisas. Sin embargo, desde C# 10, se mejoraron estas advertencias para ser más precisas.

Solución: Se ajustó la regla `expression` para mejorar las advertencias relacionadas con la asignación definida y el análisis de estado nulo.

Declaración y Asignación en Deconstrucción

Problema: La deconstrucción solo permitía declarar variables. Sin embargo, desde C# 10, se permite tanto declarar como asignar valores en la misma deconstrucción.

Solución: Se modificó la regla `deconstruction_expression` para permitir tanto declaraciones como asignaciones.

Atributo AsyncMethodBuilder

Problema: No había una forma explícita de marcar métodos como constructores de métodos asíncronos. Sin embargo, desde C# 10, se introdujo el atributo `AsyncMethodBuilder` para indicar métodos que construyen métodos asíncronos.

Solución: Se agregó soporte para el atributo `AsyncMethodBuilder`.

Atributo CallerArgumentExpression

Problema: No había una forma explícita de obtener la expresión del argumento del llamador. Sin embargo, desde C# 10, se introdujo el atributo `CallerArgumentExpression` para obtener esta información.

Solución: Se agregó soporte para el atributo `CallerArgumentExpression`.

Nuevo Formato para la Directiva `#line`

Problema: La directiva `#line` no tenía un formato flexible. Sin embargo, desde C# 10, se introdujo un nuevo formato para esta directiva que permite más flexibilidad.

Solución: Se modificó la regla `line_directive` para soportar el nuevo formato.

C# 11

Literales de Cadena Raw

Problema: Las cadenas literales no podían contener caracteres especiales sin escaparlos. Sin embargo, desde C# 11, se introdujeron los literales de cadena raw, que permiten cadenas sin necesidad de escapar caracteres especiales.

Solución: Se agregó soporte para los literales de cadena raw en la regla `string_literal`.

Soporte Genérico para Operaciones Matemáticas

Problema: Las operaciones matemáticas no podían ser genéricas. Sin embargo, desde C# 11, se introdujo el soporte genérico para operaciones matemáticas, permitiendo definir métodos que trabajen con tipos numéricos genéricos.

Solución: Se modificó la regla `type_` para permitir tipos genéricos en operaciones matemáticas.

Atributos Genéricos

Problema: Los atributos no podían ser genéricos. Sin embargo, desde C# 11, se introdujo el soporte para atributos genéricos, permitiendo definir atributos que acepten tipos genéricos.

Solución: Se modificó la regla `attribute` para permitir atributos genéricos.

Literales de Cadena UTF-8

Problema: Las cadenas literales no podían especificar codificación UTF-8 explícitamente. Sin embargo, desde C# 11, se introdujo el soporte para literales de cadena UTF-8, permitiendo especificar la codificación explícitamente.

Solución: Se agregó soporte para literales de cadena UTF-8 en la regla `string_literal`.

Salto de Línea en Expresiones Interpoladas

Problema: Las expresiones interpoladas no permitían saltos de línea dentro de la cadena. Sin embargo, desde C# 11, se permite incluir saltos de línea en expresiones interpoladas.

Solución: Se modificó la regla `interpolated_string_expression` para permitir saltos de línea.

Patrones de Lista

Problema: Los patrones no podían aplicarse a listas. Sin embargo, desde C# 11, se introdujo el soporte para patrones de lista, permitiendo comparar listas con patrones específicos.

Solución: Se agregó soporte para patrones de lista en la regla `pattern`.

Tipos Locales a Archivo

Problema: Los tipos definidos debían ser accesibles desde cualquier parte del archivo. Sin embargo, desde C# 11, se introdujo el soporte para tipos locales a archivo, permitiendo definir tipos que solo sean accesibles dentro del mismo archivo.

Solución: Se modificó la regla `class_definition` para permitir tipos locales a archivo.

Miembros Obligatorios

Problema: No había una forma explícita de marcar miembros como obligatorios. Sin embargo, desde C# 11, se introdujo el soporte para miembros obligatorios, permitiendo definir miembros que deben ser inicializados.

Solución: Se agregó soporte para miembros obligatorios en la regla `property_declaration`.

Estructuras con Valores Predeterminados

Problema: Las estructuras no tenían valores predeterminados. Sin embargo, desde C# 11, se introdujo el soporte para estructuras con valores predeterminados, permitiendo que las estructuras tengan valores por defecto.

Solución: Se modificó la regla `struct_declaration` para permitir valores predeterminados en estructuras.

Coincidencia de Patrones con `Span<char>`

Problema: Los patrones no podían aplicarse a `Span<char>`. Sin embargo, desde C# 11, se introdujo el soporte para coincidir patrones con `Span<char>`.

Solución: Se agregó soporte para patrones con `Span<char>` en la regla `pattern`.

Alcance Extendido de `nameof`

Problema: El operador `nameof` solo podía aplicarse a nombres de tipos y miembros. Sin embargo, desde C# 11, se extendió el alcance de `nameof` para permitir su uso en más contextos.

Solución: Se modificó la regla `expression` para permitir el uso extendido de `nameof`.

Tipo `IntPtr` Numérico

Problema: El tipo `IntPtr` no era numérico. Sin embargo, desde C# 11, se introdujo el soporte para que `IntPtr` sea un tipo numérico.

Solución: Se modificó la regla `type_` para permitir que `IntPtr` sea un tipo numérico.

Campos `ref` y `scoped ref`

Problema: Los campos no podían ser `ref`. Sin embargo, desde C# 11, se introdujo el soporte para campos `ref` y `scoped ref`, permitiendo definir campos que se comporten como referencias.

Solución: Se agregó soporte para campos `ref` en la regla `field_declaration`.

Mejora en la Conversión de Grupos de Métodos a Delegados

Problema: La conversión de grupos de métodos a delegados no era tan flexible. Sin embargo, desde C# 11, se mejoró esta conversión para permitir más flexibilidad en la asignación de métodos a delegados.

Solución: Se modificó la regla `expression` para mejorar la conversión de grupos de métodos a delegados.

C# 12

Constructores Primarios

Problema: Los constructores debían definirse explícitamente dentro de una clase o estructura. Desde C# 12, se permite crear constructores primarios en cualquier tipo de clase o estructura.

Solución: Se modificó la regla `class_definition` para permitir constructores primarios. Esto simplifica la definición de clases y estructuras al combinar la declaración de parámetros con la inicialización de campos.

Expresiones de Colección

Problema: Las colecciones se creaban utilizando el operador `new[]`. Desde C# 12, se introdujo una nueva sintaxis para expresiones de colección, incluyendo el elemento de propagación (`..e`).

Solución: Se agregó una nueva regla para soportar las expresiones de colección, definiendo la sintaxis de colecciones en línea y el operador de propagación (`..e`).

Arrays en Línea

Problema: Los arrays debían declararse con un tamaño fijo utilizando llaves (`{}`). Desde C# 12, se permite declarar arrays en línea dentro de estructuras.

Solución: Se modificó la regla `type_` para permitir arrays en línea, simplificando la declaración de estructuras con arrays de tamaño fijo.

Parámetros Opcionales en Expresiones Lambda

Problema: Los parámetros en expresiones lambda no podían ser opcionales. Desde C# 12, se permite el uso de parámetros opcionales en expresiones lambda.

Solución: Se actualizó la regla `explicit_anonymous_function_parameter` para soportar parámetros opcionales.

Parámetros `ref readonly`

Problema: Los parámetros en expresiones lambda no podían ser `ref readonly`. Desde C# 12, se permite el uso de parámetros `ref readonly` en expresiones lambda.

Solución: Se modificó la regla `explicit_anonymous_function_parameter` para soportar parámetros `ref readonly`.

Alias Any Type

Problema: El alias de tipo solo se podía utilizar con tipos nombrados. Desde C# 12, se permite utilizar el alias de tipo con cualquier tipo.

Solución: Se actualizó la regla `using_directive` para permitir alias de cualquier tipo.

Atributo Experimental

Problema: No había una forma explícita de marcar características experimentales. Desde C# 12, se introdujo el atributo `Experimental`.

Solución: Se agregó soporte para el atributo `Experimental`.

Bibliografía

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Mathew Eastwood, Scott Gray, Dominic Harvey, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal M. K., Josh Kratz, Kunal Malhotra, Balazs McGinnis, Sherry Moore, Derek Murray, Dave Orr, Mike Schuster, Josh Susskind, Zhuowen Tu, Vijay V., Pete Warden, Xiaoqiang Wu, and Reza Zadeh. Tensorflow: Large-scale machine learning on heterogeneous systems. *arXiv:1603.04467*, 2016. (Citado en la página 36).
- [2] Martín Abadi, Ashish Agarwal, Paul Barham, et al. Tensorflow: A system for large-scale machine learning. *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 265–283, 2016. (Citado en la página 20).
- [3] Alfred V. Aho, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 1986. (Citado en la página 14).
- [4] Alex Aiken. Moss (measure of software similarity), 1994. (Citado en las páginas 13 y 20).
- [5] Uri Alon, Shaked Brody, Omer Levy, and Eran Yahav. Code2seq: Generating sequences from structured representations of code. In *International Conference on Learning Representations (ICLR)*, 2019. (Citado en la página 16).
- [6] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. Code2vec: Learning distributed representations of code. In *Pro-*

ceedings of the ACM on Programming Languages, volume 3, pages 40:1–40:29, 2019. (Citado en la página 16).

- [7] Ethem Alpaydin. *Introduction to Machine Learning (2nd Edition)*. MIT Press, 2014.
- [8] Unspecified Author. Discovering vulnerable functions: A code similarity based approach. In *Proceedings of the 2016 International Conference on Software Engineering*, 2016. (Citado en la página 18).
- [9] Unspecified Author. Code similarity detection technique based on ast unsupervised clustering method. In *Proceedings of the 2020 International Conference on Computational Communications and Networks (ICCC)*, 2020. (Citado en la página 17).
- [10] Unspecified Author. Enhanced semantic similarity detection of program code using siamese neural network. *International Journal of Advanced Networking and Applications*, 14(2), 2022. (Citado en la página 19).
- [11] Ira D. Baxter, Andrew Yahin, Leonardo Moura, Marcelo Sant’Anna, and Lorraine Bier. Clone detection using abstract syntax trees. In *Proceedings of the International Conference on Software Maintenance (ICSM)*, pages 368–377, 1998. Propuesta de métodos para detectar clones exactos y casi coincidentes mediante subárboles de AST. (Citado en las páginas 10 y 15).
- [12] C. M. Bishop and L. J. F. H. van der Maaten. L1 distance in machine learning: Applications and analysis. *IEEE Transactions on Neural Networks*, 6(1):11–19, 1995. Explora la aplicación de la distancia L1 en el aprendizaje supervisado. (Citado en la página 36).
- [13] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006. (Citado en la página 18).
- [14] Robert S. Boyer and J. Strother Moore. A fast string searching algorithm. In *Proceedings of the 19th Annual Symposium on Switching and Automata Theory (SWAT 1978)*, pages 62–72. IEEE, 1977. (Citado en la página 13).

- [15] J. Bromley, Y. LeCun, I. Guyon, R. Shah, L. Bottou, and Y. Hu. Signature verification using a siamese time delay neural network. In *Neural Networks, 1993., IEEE International Conference on*, pages 669–674. IEEE, 1993. (Citado en la página 18).
- [16] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020. (Citado en la página 17).
- [17] H. Cai, Y. Liu, and Z. Wu. Modeling functional similarity in source code with graph-based siamese network. *ACM SIGPLAN Notices*, 2020.
- [18] Sumit Chopra, Raia Hadsell, and Yann LeCun. Learning a similarity metric discriminatively, with application to face verification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005. (Citado en la página 18).
- [19] Corinna Cortes and Vladimir Vapnik. *Support-vector networks*, volume 20. Springer, 1995. (Citado en la página 18).
- [20] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7):2121–2159, 2011. Introducción al concepto de tasas de aprendizaje adaptativas con AdaGrad. (Citado en la página 40).
- [21] M. Duracik, E. Kirsak, and P. Hrkut. Source code representations for plagiarism detection. In *Springer International Publishing AG, part of Springer Nature: CCIS 870*, pages 61–69, 2018.
- [22] Benedikt G., Matthias F., Jens K., and Helmut P. Jplag: A system for detecting software plagiarism. (Citado en la página 21).
- [23] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>. (Citado en las páginas 10 y 37).
- [24] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016. Capítulo 8, Optimización. (Citado en la página 40).

- [25] Richard H. Hahnloser, Robert S. Finkelstein, and Steven H. Seung. Sigmoid activation functions for neural networks. *Neural Computation*, 12(4):909–931, 2000. Este artículo analiza las funciones de activación sigmoideal y su aplicación en redes neuronales. (Citado en la página 36).
- [26] Charles R. Harris, K. Jarrod Millman, Stéfan van der Walt, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020. (Citado en la página 20).
- [27] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2015. (Citado en la página 19).
- [28] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. (Citado en la página 17).
- [29] Unspecified Hoq and Unspecified Shi. Detecting chatgpt-generated code submissions in a cs1 course using machine learning models. *Semantics Scholar*, 2024. (Citado en la página 17).
- [30] Timothy Kam. Analyzing and understanding software artifacts with abstract syntax trees. In *Proceedings of the 2005 International Conference on Software Engineering Research and Practice*, 2005. (Citado en la página 14).
- [31] Donald E. Knuth, James H. Morris Jr, and Vaughan R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(2):323–350, 1977. (Citado en la página 13).
- [32] Gregory Koch, Richard Zemel, and Ruslan Salakhutdinov. Siamese neural networks for one-shot image recognition. *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015. (Citado en la página 19).
- [33] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 25:1097–1105, 2012. (Citado en la página 19).

- [34] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. (Citado en la página 17).
- [35] Y. Li, F. Liu, and Y. Wang. Java source code similarity detection using siamese networks. In *Proceedings of the International Conference on Machine Learning*, 2022. (Citado en la página 19).
- [36] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008. (Citado en la página 18).
- [37] MATCOM. Domino. (Citado en la página 47).
- [38] MATCOM. Hulk. (Citado en la página 47).
- [39] MATCOM. Moogle. (Citado en la página 47).
- [40] MATCOM. Wall-e. (Citado en la página 47).
- [41] Wes McKinney. *Data Structures for Statistical Computing in Python*. Springer, New York, NY, 2010. (Citado en la página 20).
- [42] Microsoft. C# version history, 2024. (Citado en la página 24).
- [43] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013. (Citado en las páginas 18, 19, 31 y 32).
- [44] Steven S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann Publishers, 1997.
- [45] Aminu B. Muhammad, Abba Almu Hadiza Lawal, Abba Abubakar Roko, and Abdulgafar Usman. Enhanced semantic similarity detection of program code using siamese neural network. *Int. J. Advanced Networking and Applications*, 14(2):5353–5360, 2022.

- [46] Ahlijati Nuraminah and Abdullah Ammar. Damerau-levenshtein distance algorithm based on abstract syntax tree to detect code plagiarism. *Journal of Software Engineering Research and Applications*, 2019. (Citado en la página 16).
- [47] Terence Parr. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf, Raleigh, NC, 2013. (Citado en la página 20).
- [48] Adam Paszke, Sam Gross, Francisco Massa, et al. Pytorch: An imperative style, high-performance deep learning library. *Proceedings of NeurIPS*, 32, 2019. (Citado en la página 20).
- [49] Fabrice Pedregosa, Guillaume Varoquaux, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. (Citado en la página 20).
- [50] B. N. Pellin. Using support vector machines and abstract syntax trees for classifying code authorship. In *Proceedings of the 4th International Conference on Software Engineering Research and Practice*, 2004. Clasificación de autoría en código fuente utilizando SVM y AST. (Citado en la página 15).
- [51] ANTLR Project. Antlr v4. <https://github.com/antlr/antlr4>, 2025. (Citado en la página 20).
- [52] Ning Qian. On the momentum term in gradient descent learning algorithms. *Neural Networks*, 12(1):145–151, 1999. Detalles sobre el uso de Momentum. (Citado en la página 40).
- [53] Rahul Saini, Prashant Singh, Jitendra Singh, and Girdhari Sikka. Learning semantic representations of source code using word2vec. *International Journal of Advanced Computer Science and Applications*, 12(8):99, 2021. (Citado en la página 32).
- [54] Robert Smith, Mary Johnson, and Kevin Lee. A first step towards algorithm plagiarism detection using subtree comparison. In *Proceedings of the 15th International Conference on Computational Intelligence*. IEEE, 2020. (Citado en la página 16).
- [55] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to

- prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. Introducción de Dropout como técnica de regularización. (Citado en la página 37).
- [56] Ettore Merlo Thierry M. Lavoie. Levenshtein edit distance-based type iii clone detection using metric trees. 2011. (Citado en la página 16).
- [57] Wenhan Wang and Unspecified Zhang. Detecting code clones with graph neural network and flow-augmented abstract syntax tree. *ResearchGate*, 2020. (Citado en la página 17).
- [58] Xin Wang, Li Zhang, Yu Chen, et al. A weighted abstract syntax tree kernel method for source code plagiarism detection. *Journal of Software Engineering and Applications*, 2022. (Citado en la página 15).